

A  
Major Project  
On  
**AUTOMATED ANIMAL SPECIES IDENTIFICATION VIA-DEEP  
LEARNING ON BIO-ACOUSTIC RECORDINGS**

(Submitted in partial fulfilment of the requirements for the award of Degree)

**BACHELOR OF TECHNOLOGY**  
In  
**INFORMATION TECHNOLOGY**  
By

D.Himavani (227R1A1213)

A.Aniketh (227R1A1203)

M.Vishwani (227R1A1235)

P.Somnath (217R1A12B4)

Under the Guidance of

**G. Shankar**

(Assistant Professor)



**DEPARTMENT OF INFORMATION TECHNOLOGY**

**CMR TECHNICAL CAMPUS**

**UGC AUTONOMOUS**

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi)

Recognized Under Section 2(f) & 12(B) of the UGC Act. 1956,

Kandlakoya (V), Medchal Road, Hyderabad-501401.

**May, 2026.**

# DEPARTMENT OF INFORMATION TECHNOLOGY



## CERTIFICATE

This is to certify that the project entitled “**AUTOMATED ANIMAL SPECIES IDENTIFICATION VIA DEEP LEARNING ON BIO-ACOUSTIC RECORDINGS**” being submitted by **D.Himavani (227R1A1213), A.Aniketh (227R1A1203), M.Vishwani (227R1A1235) & P.Somnath (217R1A12B4)** in partial fulfilment of the requirements for the award of the degree of B.Tech in Information Technology to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

**G. Shankar**  
**Assistant Professor**  
**INTERNAL GUIDE**

**HOD**

**Dr. A. Raji Reddy**  
**DIRECTOR**

**Signature of External Examiner**

**Submitted for viva voice Examination held on \_\_\_\_\_**

## ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project, we take this opportunity to express our profound gratitude and deep regard to our guide **G. Shankar**, Assistant Professor for his exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by him shall carry us a long way in the journey of life on which we are about to embark.

We take this opportunity to extend our heartfelt appreciation to the Project Review Committee (PRC) Coordinators— **Dr. J. Narasimharao, Mr. Md. Sajid Pasha, Mr. K. Praveen Kumar, Mr. D. Krishna Kumar, Mr. Y. Satyam and Mr. K. Srinu** —for their unwavering support, insightful guidance, and valuable inputs, which played a crucial role in steering this project through its various stages.

Our sincere appreciation also goes to our **Head of the Department**, for encouragement and continuous support in ensuring the successful completion of our project.

We are deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, we extend our profound gratitude to Sri. **Ch. Gopal Reddy**, Chairman, Smt. **C. Vasantha Latha**, Secretary and Sri. **C. Abhinav Reddy**, Vice-Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to our progress.

We also acknowledge and appreciate the guidance and assistance provided by the faculty and staff of **CMR Technical Campus**, whose contributions have been invaluable in bringing this project to fruition.

Lastly, we sincerely thank our families for their unwavering support and encouragement. We also extend our gratitude to the teaching and non-teaching staff of CMR Technical Campus for their guidance and assistance. Their contributions, along with the support of everyone who helped directly or indirectly, have been invaluable in the successful completion of this project.

**D.Himavani(227R1A1213)**  
**A.Aniketh(227R1A1203)**  
**M.Vishwani(227R1A1235)**  
**P.Somnath(217R1A12B4)**

## **VISION AND MISSION**

### **INSTITUTE VISION:**

To Impart quality education in serene atmosphere thus strive for excellence in Technology and Research.

### **INSTITUTE MISSION:**

1. To create state of art facilities for effective Teaching- Learning Process.
2. Pursue and Disseminate Knowledge based research to meet the needs of Industry & Society.
3. Infuse Professional, Ethical and Societal values among Learning Community.

### **DEPARTMENT VISION:**

To produce technically proficient and skilled graduates in Information Technology and to prepare them to meet the needs of industry and society in a holistic learning environment.

### **DEPARTMENT MISSION:**

1. To prepare students with the essential knowledge and contemporary abilities for solving emerging technological problems.
2. To train the students on cutting – edge computing facilities with industry interaction to enhance in practical exposure to real world situations.
3. To enhance the leadership qualities and team spirit with social values to serve the society.

## ABSTRACT

The rapid advancement of wildlife monitoring and ecological research has created a strong demand for intelligent systems capable of automatically identifying animal species using non-invasive methods. Bioacoustics recordings, which capture animal vocalizations, offer a reliable and scalable alternative to visual monitoring, especially in dense forests and low-visibility environments. With the growth of machine learning and audio signal processing, automated analysis of animal sounds has become a promising research direction for biodiversity conservation and environmental surveillance. Despite these advances, accurate animal species identification remains a challenging problem due to overlapping vocal patterns, background noise, varying recording conditions, and class imbalance in real-world datasets. Traditional systems largely rely on manual inspection or standalone machine learning classifiers such as Support Vector Machines or Decision Trees. These approaches suffer from limited generalization capability, poor robustness to complex acoustic variations, and reduced accuracy in multi-class classification scenarios. Additionally, most traditional systems lack interactive user interfaces and secure access control, making them unsuitable for practical deployment. To address these limitations, this project proposes an Automated Animal Species Identification System using Bioacoustics Recordings with a Hybrid SVM + Light GBM (Soft Voting) model. The system performs systematic audio preprocessing and extracts rich handcrafted features including MFCCs, chroma, mel-spectrogram, spectral contrast, tonnetz, and energy-based descriptors. A hybrid ensemble learning approach is employed to combine the strengths of margin-based learning and gradient boosting, resulting in improved classification accuracy and robustness. The significance of the proposed system lies in its high classification performance, achieving 98.33% accuracy and 99.26% ROC–AUC, substantially outperforming traditional classifiers. By providing a reliable, scalable, and user-friendly solution, the system reduces human effort and enhances efficiency in wildlife monitoring, biodiversity assessment, and bioacoustic research. This work demonstrates the effectiveness of ensemble learning for complex audio classification tasks and offers a strong foundation for future intelligent environmental monitoring systems.

| FIGURE NO           | LIST OF FIGURES<br>FIGURE NAME                                                                               | PAGE NO |
|---------------------|--------------------------------------------------------------------------------------------------------------|---------|
| <b>Figure 3.1</b>   | Project Architecture of Automated animal species Identification via deep learning on Bio-acoustic recordings | 13      |
| <b>Figure 3.3</b>   | Dataflow Diagram of Automated animal species Identification via deep learning on Bio-acoustic recordings     | 15      |
| <b>Figure 3.4</b>   | Class Diagram of Automated animal species Identification via deep learning on Bio-acoustic recordings        | 16      |
| <b>Figure 3.5</b>   | Use case Diagram of Automated animal species Identification via deep learning on Bio-acoustic recordings     | 17      |
| <b>Figure 3.6</b>   | Sequence Diagram of Automated animal species Identification via deep learning on Bio-acoustic recordings     | 18      |
| <b>Figure 3.7</b>   | Activity Diagram of Automated animal species Identification via deep learning on Bio-acoustic recordings     | 19      |
| <b>Figure 4.1.1</b> | Screenshot showing a folder required for automated animal species Identification system                      | 25      |
| <b>Figure 4.1.2</b> | Screenshot showing Dataset folder which contains 6 different animal species.                                 | 25      |
| <b>Figure 4.1.3</b> | Screenshot showing test audio recordings of different animal species                                         | 26      |

|                    |                                                                  |    |
|--------------------|------------------------------------------------------------------|----|
| <b>Figure 5.1</b>  | Admin login interface of automated species Identification system | 43 |
| <b>Figure 5.2</b>  | Dataset Upload and animal class detection Interface              | 44 |
| <b>Figure 5.3</b>  | Audio feature Extraction Completion Output Screen                | 45 |
| <b>Figure 5.4</b>  | Audio Feature Extraction output Confirmation                     | 46 |
| <b>Figure 5.5</b>  | Existing SVM Confusion matrix                                    | 47 |
| <b>Figure 5.6</b>  | Existing SVM ROC Curve                                           | 48 |
| <b>Figure 5.7</b>  | Existing Decision Tree Confusion matrix                          | 49 |
| <b>Figure 5.8</b>  | Existing Decision Tree ROC Curve                                 | 49 |
| <b>Figure 5.9</b>  | Hybrid SVM+LGBM confusion Matrix                                 | 50 |
| <b>Figure 5.10</b> | Hybrid SVM+LGBM Curve                                            | 51 |

|                    |                                                            |    |
|--------------------|------------------------------------------------------------|----|
| <b>Figure 5.11</b> | User Login Interface for Prediction Access                 | 52 |
| <b>Figure 5.12</b> | Audio Waveform Visualization with predicted animal species | 52 |



## LIST OF TABLES

| TABLE NO    | TABLE NAME                | PAGE NO |
|-------------|---------------------------|---------|
| Table 6.2.1 | Dataset class description | 54      |
| Table 6.2.2 | Classification            | 54      |

# TABLE OF CONTENTS

|                                            |    |
|--------------------------------------------|----|
| <b>ABSTRACT</b>                            | i  |
| <b>LIST OF FIGURES</b>                     | ii |
| <b>LIST OF TABLES</b>                      | v  |
| <b>1. INTRODUCTION</b>                     | 1  |
| 1.1 PROJECT PURPOSE                        | 1  |
| 1.2 PROJECT FEATURES                       | 2  |
| <b>2. LITERATURE SURVEY</b>                | 3  |
| 2.1 REVIEW OF RELATED WORK                 | 7  |
| 2.2 DEFINITION OF PROBLEM STATEMENT        | 8  |
| 2.3 EXISTING SYSTEM                        | 8  |
| 2.4 PROPOSED SYSTEM                        | 10 |
| 2.5 OBJECTIVES                             | 11 |
| 2.6 HARDWARE & SOFTWARE REQUIREMENTS       | 12 |
| 2.6.1 HARDWARE REQUIREMENTS                | 12 |
| 2.6.2 SOFTWARE REQUIREMENTS                | 12 |
| <b>3. SYSTEM ARCHITECTURE &amp; DESIGN</b> | 12 |
| 3.1 PROJECT ARCHITECTURE                   | 13 |
| 3.2 DESCRIPTION                            | 14 |
| 3.3 DATA FLOW DIAGRAM                      | 15 |
| 3.4 CLASS DIAGRAM                          | 16 |
| 3.5 USE CASE DIAGRAM                       | 17 |
| 3.6 SEQUENCE DIAGRAM                       | 18 |
| 3.7 ACTIVITY DIAGRAM                       | 19 |
| <b>4. IMPLEMENTATION</b>                   | 20 |
| 4.1 ALGORITHMS USED                        | 20 |
| 4.2 SAMPLE CODE                            | 28 |
| <b>5. RESULTS &amp; DISCUSSION</b>         | 43 |
| <b>6. VALIDATION</b>                       | 53 |
| 6.1 INTRODUCTION                           | 53 |
| 6.2 TEST CASES                             | 54 |
| 6.2.1 DATASET CLASS DESCRIPTION            | 54 |
| 6.2.2 CLASSIFICATION                       | 54 |
| <b>7. CONCLUSION &amp; FUTURE ASPECTS</b>  | 55 |
| 7.1 PROJECT CONCLUSION                     | 55 |
| 7.2 FUTURE ASPECTS                         | 56 |
| <b>8. BIBLIOGRAPHY</b>                     | 57 |
| 8.1 REFERENCES                             | 57 |
| 8.2 GITHUB LINK                            | 58 |

# **1.INTRODUCTION**

## **1.INTRODUCTION**

Automatic acoustic classification also referred to as audio or sound classification, involves the detection or recognition of sound using audio informatics for storage and retrieval, and machine learning techniques for autonomous classification. Bioacoustics is the branch of acoustics that is concerned with sounds produced by or affecting living organisms. Bioacoustics is often used in acoustic sensing to monitor biodiversity, especially in visually inaccessible areas. Animal acoustic emissions contain species-specific information that reflects the character and behavior of different living organisms. There are three main application areas of bioacoustics. The first focuses on the classification and analysis of sounds vocalized by different animal species. Its primary aim is to identify sounds that characterize species in different behavioral contexts. The second is concerned with integrating sound signals vocalized by animals with behavioral contexts to understand how the sounds affect the behavior and emotions of the receiver. The third explores the production mechanisms used in sound vocalization processes. The survey presented in this paper explores how current research in automated bioacoustics classification differs from traditional acoustic classification with respect to the techniques used and application areas.

### **1.1 PROJECT PURPOSE**

The purpose of this project is to develop an automated system capable of identifying animal species by analyzing their bio-acoustic signals using deep learning techniques. Many animals communicate through vocalizations such as calls, songs, and other sounds, which can serve as unique acoustic signatures for species identification.

Traditional wildlife monitoring methods rely heavily on manual observation or expert analysis of audio recordings, which are time-consuming, labor-intensive, and prone to human error. This project aims to address these limitations by applying deep learning models to automatically detect and classify animal species from environmental audio recordings.

By processing bio-acoustic data and training neural networks on labeled sound datasets, the system can learn patterns associated with specific species. The developed model will help

in faster and more accurate species recognition, enabling large-scale monitoring of wildlife populations.

Ultimately, the project contributes to ecological research, biodiversity monitoring, and conservation efforts by providing a scalable and efficient tool for tracking animal presence and behavior in natural habitats.

## **1.2 PROJECT FEATURES**

The Automated Animal Species Identification via Deep Learning on Bio-Acoustic Recordings project focuses on identifying animal species by analyzing their vocal sounds using deep learning techniques. The system processes audio recordings, converts them into spectrograms, and uses trained neural networks to detect patterns in animal calls. This automated approach helps researchers monitor wildlife more efficiently, reduces manual effort, and improves the accuracy of species identification for biodiversity conservation.

### **Key Features:**

- Automated detection of animal species from audio recordings
- Use of deep learning models for accurate classification
- Conversion of audio signals into spectrograms for analysis
- Ability to process large amounts of bio-acoustic data
- Supports wildlife monitoring and biodiversity conservation efforts

## **2.LITERATURE SURVEY**

## 2.LITERATURE SURVEY

Mutanu, et al. examined bioacoustics classification alongside general acoustics to provide a representative picture of the research landscape. The survey reviewed 124 studies spanning eight years of research. The survey identifies the key application areas in bioacoustics research and the techniques used in audio transformation and feature extraction. The survey also examines the classification algorithms used in bioacoustics systems.

Karaaslan, et al proposed an automated system for detecting and classifying animal vocalizations, enhancing efficiency in behaviour analysis. The system uses a preprocessing step to segment relevant sound regions from audio recordings, followed by feature extraction using Short-Time Fourier Transform (STFT), Mel-frequency cepstral coefficients (MFCCs), and linear-frequency cepstral coefficients (LFCCs). These features are input into convolutional neural network (CNN) classifiers to evaluate performance. Experimental results demonstrate the effectiveness of different CNN models and feature extraction methods, with AlexNet, DenseNet, EfficientNet, ResNet50, and ResNet152 being evaluated.

Aliane, et al conducted a comprehensive literature review to examine the recent advancements in drone and AI systems for wildlife monitoring, focusing on two critical dimensions: Methodologies, algorithms, and applications, analyzing the AI techniques employed in wildlife monitoring, including their operational frameworks and real-world implementations. Challenges and opportunities, identifying current limitations, including technical hurdles and regulatory constraints, as well as exploring the untapped potential in drone and AI integration to enhance wildlife monitoring and conservation efforts.

Oswald, et al. contributed towards filling this gap. Instead of a classical list of “dos” and “don’ts”, we review some key papers which, we believe, embody best practices in several bioacoustic subfields. In the first three case studies, they discuss how bioacoustics can help identify the ‘who’, ‘where’ and ‘how many’ of animals within a given ecosystem. Specifically, we review cases in which bioacoustic methods have been applied with success to draw inferences regarding species identification, population structure, and biodiversity.

Sharma, et al. reviewed showed a significant rise in the utilization of AI techniques in wildlife acoustic monitoring over this period, with birds ( $N = 26$ ) gaining the most popularity, followed by mammals ( $N = 12$ ). The most commonly used AI algorithm in this field was Convolutional Neural Network, which was found to be more accurate and beneficial than previous categorization methods in acoustic wildlife monitoring. This highlights the potential for AI to play a crucial role in advancing our understanding of wildlife populations .

Chao, et al. presented the implementation of artificial intelligence (AI) for classification of frogs in symmetry of the bioacoustics spectral by using the feedforward neural network approach (FNN)

support vector machine (SVM). Recently, the symmetry concept has been applied in physics, and in mathematics to help make mathematical models tractable to achieve the best learning performance. Owing to the symmetry of the bioacoustics spectral, feature extraction can be achieved by integrating the techniques of Mel-scale frequency cepstral coefficient (MFCC) and mentioned machine learning algorithms, such as SVM, neural network, and so on.

Manikandan, et al. revealed compelling evidence for edge computing deployment via TinyML frameworks, addressing critical scalability challenges in commercial poultry environments characterized by acoustic complexity and computational constraints. Advanced applications spanning emotion recognition, disease detection, and behavioral phenotyping demonstrate unprecedented potential for real-time welfare assessment. Through rigorous bibliometric co-occurrence mapping and thematic clustering analysis, this review exposes persistent methodological bottlenecks: dataset standardization deficits, evaluation protocol inconsistencies, and algorithmic interpretability limitations.

Ratnayake, et al. reviewed systematically explores the application of machine learning (ML) techniques in bee species determination, shedding light on the transformative potential of ML in entomology. Conducting a keyword-based search in the Scopus and Web of Science databases with manual screening resulted in 26 relevant publications. Focusing on shallow and deep learning studies, our analysis reveals a significant inclination towards deep learning, particularly post-2020, underscoring its ability to handle complex, high-dimensional data for accurate species identification.

Trapanotto, et al. evaluated the performance of three pretrained CNNs (VGG16, ResNet50, and AlexNet) on a small, publicly available lion roar dataset containing approximately 150 samples taken from five male lions. Each of these networks is retrained on eight representations of the samples: MFCCs, spectrogram, and Mel spectrogram, along with several new ones, such as VGGish and stockwell, and those based on the recently proposed LM spectrogram. The performance of these networks, both individually and in ensembles, is analyzed and corroborated using the Equal Error Rate and shown to surpass previous classification attempts on this dataset; the best single network achieved over 95% accuracy and the best ensembles over 98% accuracy.

Dewmini, et al. addressed elephant sound classification utilizing raw audio processing.



Our focus lies on exploring lightweight models suitable for deployment on resource-constrained edge devices, including Mobile Net, YAMNET, and Raw Net, alongside introducing a novel model termed ElephantCallerNet. Notably, our investigation reveals that the proposed ElephantCallerNet achieves an impressive accuracy of 89% in classifying raw audio directly without converting it to spectrograms. Leveraging Bayesian optimization techniques, we fine-tuned crucial parameters such as learning rate, dropout, and kernel size, thereby enhancing the model's performance.

Alberti, et al. conducted a comparative analysis of traditional and innovative methods involving the recording and analysis of buzzing sounds emitted by flying insects. Insect abundance, reported as counts for traditional methods gathered by pan traps, and average buzzes per hour, measured by acoustic IoT (Internet of Things) sensors, were correlated. This finding suggests that acoustic automatic monitoring shows the potential to circumvent the limitations of conventional methods, emphasising continuous, non-intrusive monitoring for robust data acquisition.

Xie, et al. investigated various transfer learning models for extracting deep cascaded features with multi-view spectrograms. They found that deep cascade features being extracted from VGG16 and MobileNetV2 can achieve the best performance using repeat-based spectrogram.

Ijaz, et al, presented a few-shot learning (FSL) method incorporating transductive inference and data augmentation to address the issues of too few labeled events and small volumes of recordings. Here, transductive inference iteratively alters class prototypes and feature extractors to seize essential patterns, whereas data augmentation applies SpecAugment on Mel spectrogram features to augment training data. The proposed approach is evaluated by using the Detecting and Classifying Acoustic Scenes and Events (DCASE) 2022 and 2021 datasets.

Hu, et al. proposed an innovative deep learning model, VBSNet, designed to recognize and classify a variety of biological calls, including those of endangered western black-crested gibbons and certain bird species. The model incorporates the image feature extraction capability of the VGG16 convolutional network, the sequence modeling capability of bi-directional LSTM, and the feature selection capability of the SE attention module, realizing the multimodal fusion of image, sequence and attention information.

Nanni, et al. demonstrated that the proposed approach performs competitively (without ad-hoc optimization of the clustering methods) on both animal vocalization datasets. To further demonstrate the power of the proposed system, the best standalone approach is also evaluated.

Alipek, et al. presented an automated analysis pipeline applied to a large dataset recorded over a period of two years in a wind test field. This work proposes various statistical methods based on convolutional neural networks and clustering techniques to examine the relationship between background noise and bat echolocation sounds. In addition, the methodology performs classification at both the genus and species levels, with a high accuracy for most bat classes.

Kim, et al. proposed a two-step sound augmentation scheme using a class-conditional GAN. First, common features are learned from all classes of animal sounds, and multiple classes of animal sounds are generated based on the features that consider both waveforms and spectrograms using class-conditional GAN. Second, we select data from the generated data based on the confidence of the pretrained ASC model to improve classification performance. Through experiments, we show that the proposed method improves the accuracy of the basic ASC model by up to 18.3%, which corresponds to a performance improvement of 13.4% compared to the second-best augmentation method.

Pandeya, et al. emphasized the use of two types of learned features from deep neural networks; one from a pre-trained convolutional neural net (CNN) on music data by transfer learning and the other from unsupervised convolutional deep belief network that is (CDBN) solely trained on a collected set of cat sounds. In addition to conventional GAP, we propose an effective pooling method called FDAP to explore a number of meaningful features. In FDAP, the frequency dimension is roughly divided and then the average pooling is applied in each division. For the classification, we exploited five different machine learning algorithms and an ensemble of them.

Nieto-Mora, et al. proposed a non-supervised framework using autoencoders to extract soundscape features. We applied this framework to a dataset from Colombian landscapes captured by 31 audiomoth recorders. Our method generates clusters based on autoencoder features and represents cluster information with prototype spectrograms using centroid features and the decoder part of the neural network. Our analysis provides valuable insights into the distribution and temporal patterns of various sound compositions within the study area.

## 2.1 REVIEW OF RELATED WORK

Several research studies have focused on the automatic identification of animal species using bio-acoustic recordings and machine learning techniques. Traditional approaches mainly relied on manual feature extraction methods such as Mel-Frequency Cepstral Coefficients (MFCC), spectrogram analysis, and other acoustic features to represent animal sounds. These features were then classified using machine learning algorithms such as Support Vector Machines (SVM), k-Nearest Neighbors (KNN), and Decision Tree Classifiers (DTC). Although these methods provided a basic framework for animal sound classification, their performance was often limited due to variations in environmental noise, overlapping calls, and the complexity of bio-acoustic signals.

In several earlier studies, Support Vector Machine (SVM) models were widely used because of their ability to handle high-dimensional acoustic features. However, SVM-based systems often struggled with large and complex datasets, resulting in relatively low classification accuracy. Similarly, Decision Tree Classifiers (DTC) were applied for animal sound recognition due to their simplicity and interpretability. While DTC models performed better than some traditional classifiers, they were prone to overfitting and could not effectively capture complex patterns present in bio-acoustic recordings. As reflected in experimental comparisons, existing SVM and DTC approaches achieved relatively low performance with accuracies of 11.66% and 31.66%, respectively.

To overcome these limitations, recent research has explored hybrid and ensemble learning methods that combine multiple algorithms to improve classification performance. Ensemble models such as gradient boosting techniques have shown promising results in handling complex datasets and improving predictive accuracy. In this project, a Hybrid SVM + LightGBM model is proposed, which leverages the strengths of SVM for feature representation and LightGBM for efficient gradient boosting-based classification. The experimental results demonstrate a significant improvement in performance, achieving 98.33% clearly outperforming traditional models. This highlights the effectiveness of hybrid machine learning approaches in bio-acoustic species identification and supports their potential application in wildlife monitoring and biodiversity conservation.

## **2.2 DEFINITION OF PROBLEM STATEMENT**

With increasing acoustic data from forests, conservation areas, and research sites, there is a pressing need to automate the identification of animal species from audio recordings. Manual annotation is time-consuming, error-prone, and often requires expert knowledge of animal vocalizations. Moreover, the presence of overlapping environmental noises, varying recording qualities, and the need to process data in real-time further complicate the classification task. Traditional machine learning approaches struggle to handle the variability and complexity of bioacoustic signals, especially in noisy or multi-species environments. These systems often rely on handcrafted features, which may fail to generalize well across species or environments. There is also a lack of scalable tools that can process large volumes of real-time audio data and produce accurate and interpretable results suitable for practical deployment. Thus, the core problem lies in building a robust and intelligent system capable of extracting meaningful features from audio signals and accurately classifying animal species despite background noise and data imbalance. This system should work efficiently on both pre-recorded datasets and real-time audio inputs.

## **2.3 EXISTING SYSTEM**

In traditional wildlife monitoring systems, animal species identification was primarily conducted using manual observation techniques or passive recording tools. Field biologists and ecologists would use visual sightings, trap cameras, and handheld microphones to document the presence and behavior of animals. Audio recordings, if collected, were manually analyzed by trained personnel who relied on auditory patterns or spectrogram inspection to recognize species-specific vocalizations. The recordings were often replayed and studied frame-by-frame using software like Raven Pro or Praat for frequency and temporal pattern matching.

The data collected through these manual methods were limited in scope and often lacked temporal continuity due to resource constraints. Most of the analysis was offline, with field researchers taking back the recordings for further study at a central facility. Additionally, the extraction of meaningful features from sound files was largely based on handcrafted features like pitch, duration, and loudness, which required deep domain expertise and yielded inconsistent results under noisy environments. Machine learning, when applied in older systems, was often based on classical models like k-Nearest Neighbors (k-NN), Hidden Markov Models (HMMs), or basic Decision Trees trained on manually engineered audio features. These models lacked the capacity to handle the complex variability of animal vocalizations across species, seasons, or recording devices. Furthermore, they often failed to scale or generalize well to large datasets, especially those with unbalanced class distributions.

## **Limitations of Existing System**

Traditional systems rely heavily on manual annotation, which is time-consuming, labor-intensive, and requires expert intervention.

- Lack of automation prevents real-time monitoring and rapid response in critical conservation scenarios.
- Handcrafted feature extraction methods often perform poorly under noisy or overlapping sound conditions.
- The identification accuracy is highly dependent on the skill and auditory ability of the analyst, introducing subjectivity.
- Spectrogram interpretation without AI support may miss hidden patterns in large datasets.
- Legacy machine learning models are not robust enough to handle inter-species variation and low-quality recordings.
- Data collected manually is often sparse, fragmented, and lacks timestamped metadata, affecting longitudinal studies.
- The traditional approach does not support on-site analysis, making it unsuitable for real-time applications.
- Most methods are not scalable or adaptive to new species without complete retraining or rule adjustment. Environmental and background noises are not effectively filtered, leading to high false-positive rates in classification.

## 2.4 PROPOSED SYSTEM

- **Bioacoustics Recording Dataset:** Environmental animal sound recordings are collected and used as the input dataset for training and testing the species identification system.
- **Data Preprocessing and Augmentation:** The audio recordings are cleaned using noise reduction techniques and augmented through signal shifting, then divided into training and testing sets in an 80/20 ratio.
- **Feature Extraction Pipeline:** Important acoustic information is extracted from the audio using both handcrafted features and deep learning-based representations. Traditional audio features such as MFCC, chroma, and spectral contrast are extracted to capture frequency and tonal characteristics of animal calls.
- **Spectrogram Generation:** Audio signals are converted into visual representations using Mel spectrograms and Short-Time Fourier Transform (STFT) to analyze frequency variations over time.
- **Deep Spectrogram Embedding:** A deep learning model processes the spectrogram images to learn complex feature representations automatically.
- **Deep Learning and Hybrid Model Training:** The extracted features are used to train a deep transfer learning classifier and a hybrid ensemble model combining Support Vector Classifier (SVC) and Light Gradient Boosting Machine (LGBM).
- **Model Evaluation:** The system performance is evaluated using metrics such as confusion matrix, accuracy, F1-score, and AUC.
- **Real-Time Prediction:** The trained model performs real-time classification of new audio recordings and generates evaluation outputs like ROC curves and metrics tables.

### Advantages of the Proposed System:

- Automated feature extraction and classification significantly reduce the time and effort required compared to manual methods, enabling faster processing of large datasets.
- The hybrid SVC + LGBM ensemble improves classification accuracy and robustness, even for species with similar acoustic patterns.
- Real-time audio playback and waveplot visualization provide immediate feedback, enhancing interpretability and usability for ecological studies.

- The system is scalable and adaptable, supporting continuous monitoring and large-scale biodiversity assessment without human intervention.

## 2.5 OBJECTIVES

- To develop an intelligent system capable of automatically identifying animal species from bio-acoustic recordings using deep learning techniques.
- To collect and preprocess animal sound datasets by removing noise, segmenting audio clips, and converting recordings into suitable formats such as spectrograms for analysis.
- To extract meaningful acoustic features from the recordings using pre-trained deep learning models to improve classification accuracy.
- To design and implement deep learning algorithms that can effectively learn patterns in animal sounds and distinguish between different species.
- To integrate ensemble classification methods that combine multiple model predictions to improve the reliability and performance of species identification.
- To develop a user-interactive platform that allows users to upload bio-acoustic recordings easily for automated analysis.
- To enable real-time or near real-time sound analysis and classification for quick identification of animal species.
- To evaluate the performance of the proposed system using metrics such as accuracy, precision, recall, and F1-score.

## **2.6 HARDWARE & SOFTWARE REQUIREMENTS**

### **2.6.1 HARDWARE REQUIREMENTS:**

Hardware interfaces specifies the logical characteristics of each interface between the software product and the hardware components of the system. The following are some hardware requirements,

- Processor: Intel i5
- Memory: 1 TB (storage)
- RAM: 8 GB

### **2.6.2 SOFTWARE REQUIREMENTS:**

Software Requirements specifies the logical characteristics of each interface and software components of the system. The following are some software requirements,

- Python 3.7.6
- Operating system: Windows
- Frontend: Tkinter
- Database: TinyDB



# **3.SYSTEM ARCHITECTURE AND DESIGN**

### 3.SYSTEM ARCHITECTURE & DESIGN

The system architecture for automated animal species identification using bioacoustic recordings is designed to process environmental audio data and accurately classify animal sounds using deep learning techniques. It integrates several stages, including data preprocessing, feature extraction, model training, and prediction, to transform raw audio recordings into meaningful information. By combining handcrafted acoustic features with deep spectrogram-based features and hybrid machine learning models, the architecture enhances the accuracy and efficiency of identifying different animal species from their vocalizations.

#### 3.1PROJECT ARCHITECTURE

The diagram shows the pipeline of a proposed system that automatically identifies animal species from bio-acoustic recordings using deep learning and hybrid machine learning models.

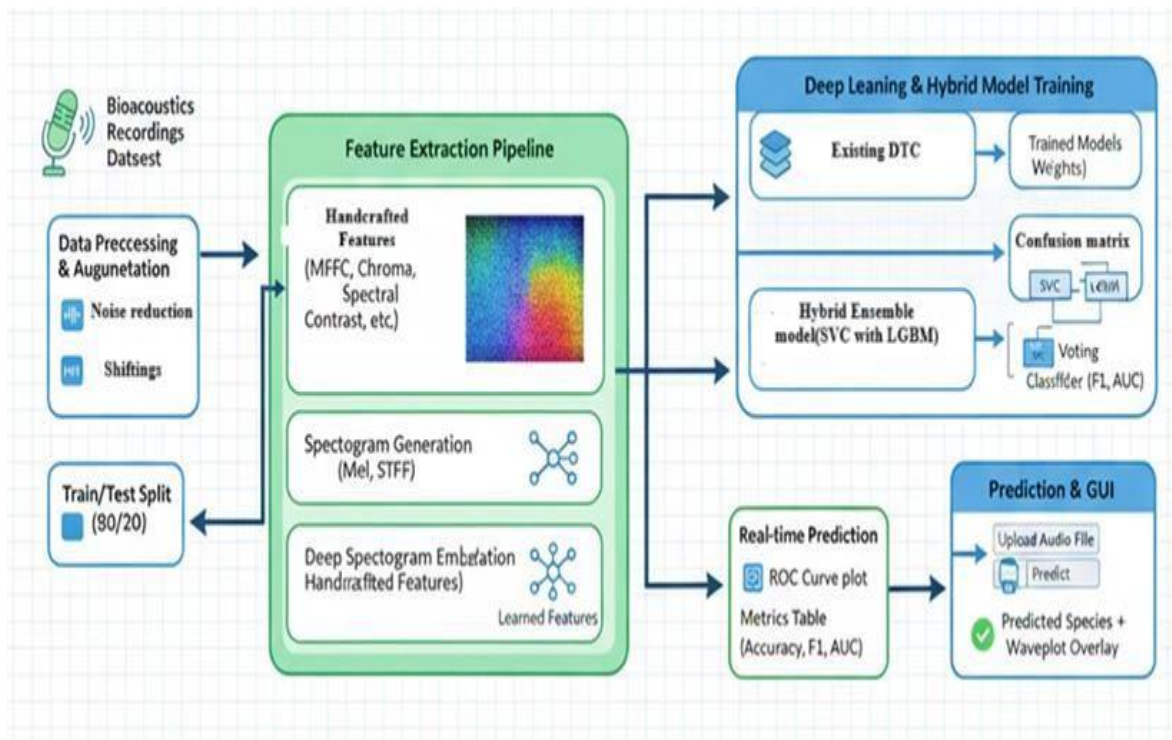


Fig 3.1: System Architecture for Automated animal species Identification via deep learning on bio-acoustic features

## 3.2 DESCRIPTION

**Dataset Acquisition:** Acoustic recordings are collected and organized into class-wise folders representing different motor operating or fault conditions.

**Audio Preprocessing:** Input audio signals are converted to mono format, normalized, and resampled to a standard sampling rate to ensure consistency across the dataset.

**Deep Feature Extraction:** High-level acoustic embeddings are extracted from raw waveforms using a pretrained deep neural audio model, capturing both temporal and spectral characteristics.

**Feature Caching:** Extracted feature vectors and corresponding labels are stored to avoid redundant computation during repeated experiments.

**Train–Test Splitting:** The dataset is divided into training and testing subsets to enable unbiased performance evaluation.

**Model Training:** Multiple classifiers, including conventional and ensemble-based models, are trained using the extracted deep audio features.

**Performance Evaluation:** The trained models are evaluated using standard metrics such as accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC curves.

**Fault Prediction:** New acoustic samples are analysed to predict the corresponding motor condition, supported by waveform visualization for interpretability.

**User Interaction:** A graphical user interface enables easy execution of each stage, making the system suitable for both research experimentation and practical deployment.

## 3.3 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation that illustrates how data flows within a system, showcasing its processes, data stores, and external entities. It is a vital tool in system analysis and design, helping stakeholders visualize the movement of information, identify inefficiencies, and optimize workflows.

A Data Flow Diagram (DFD) illustrates how data moves through a system, showing processes, data stores, external entities, and the flow of information between them. Unlike UML diagrams, which focus on system structure or behavior, DFDs emphasize the transformation and movement of data, making them ideal for understanding how information is processed and stored. They are particularly useful for breaking down complex systems into manageable, data-centric processes.

The data flow diagram illustrates the process of automatically identifying animal species from an audio recording. First, the user uploads an audio file in .wav format, which is read by the system as a raw waveform. The audio signal is then processed through a feature extraction stage using YAMNet which converts the sound into a 1024-dimensional feature vector representing important acoustic characteristics. These extracted features are passed to two classification models: Support Vector Machine and LightGBM, which each produce probability scores for different species. The outputs from both models are combined using an ensemble soft voting method to determine the most probable species. Finally, the system displays the predicted animal species to the user as the output.

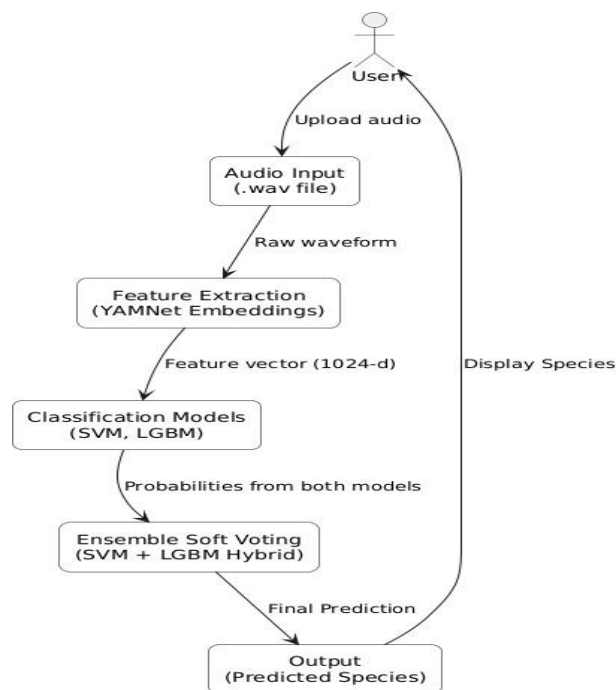
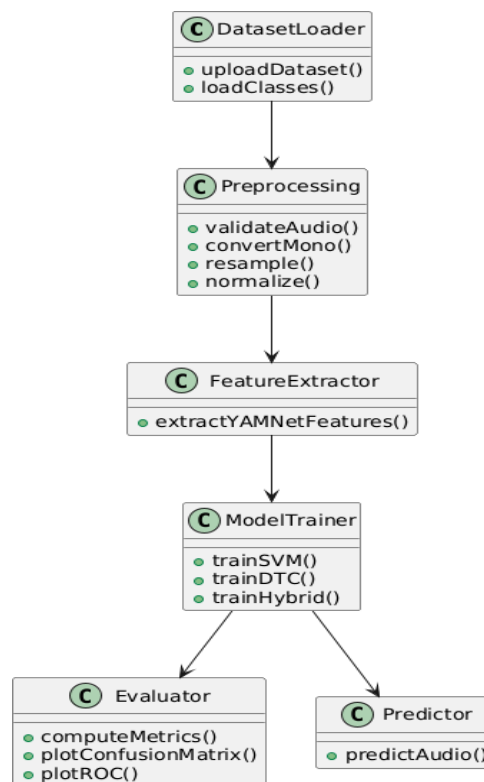


Figure 3.3: Data Flow Diagram of automated animal species Identification via deep learning on bio-acoustic recordings

### 3.4 CLASS DIAGRAM

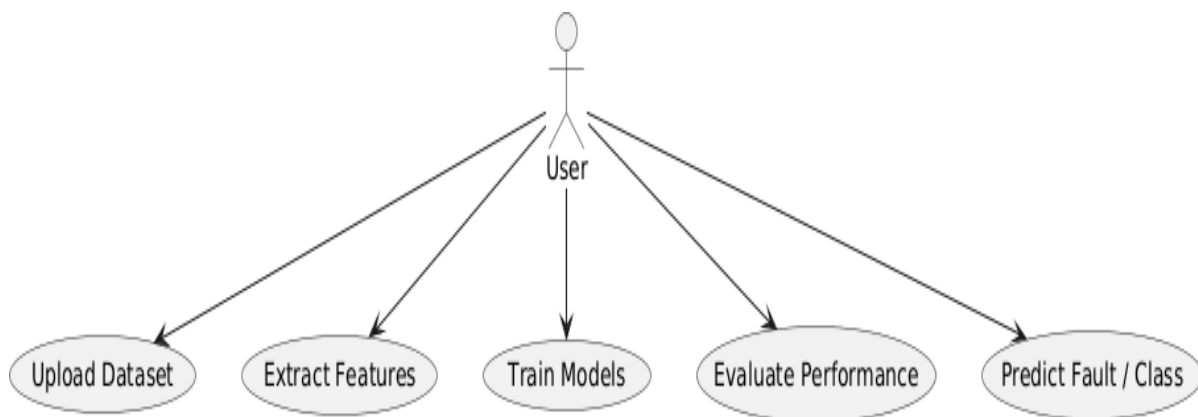
This class diagram represents an end-to-end audio classification pipeline organized into modular components. It begins with the `DatasetLoader`, which handles uploading datasets and loading class labels, followed by `Preprocessing`, where audio data is validated, converted to mono, resampled, and normalized to ensure consistency. The processed data is then passed to the `FeatureExtractor`, which derives meaningful features (e.g., using YAMNet for model input). The `ModelTrainer` uses these features to train different models such as SVM, Decision Tree, or a hybrid approach. Finally, the pipeline splits into two outcomes: the `Evaluator`, which assesses model performance through metrics, confusion matrices, and ROC curves, and the `Predictor`, which applies the trained model to make predictions on new audio data.



**Figure 3.4:**Class Diagram for Automated animal species Identification

### 3.5 USE CASE DIAGRAM

This use case diagram shows how a single User interacts with an automated system for animal species identification using bio-acoustic recordings. The user initiates the workflow by uploading a dataset of audio recordings, which the system then processes in stages. They can trigger feature extraction, where meaningful acoustic features (e.g., spectrograms or embeddings) are derived from the recordings, followed by training models using deep learning techniques to recognize species patterns. After training, the user can evaluate performance through metrics like accuracy or confusion matrices to assess how well the model performs. Finally, the system allows the user to predict faults/classes, meaning it can classify new, unseen audio recordings into specific animal species. Overall, the diagram highlights a streamlined interaction where the user controls each major step of the machine learning pipeline.

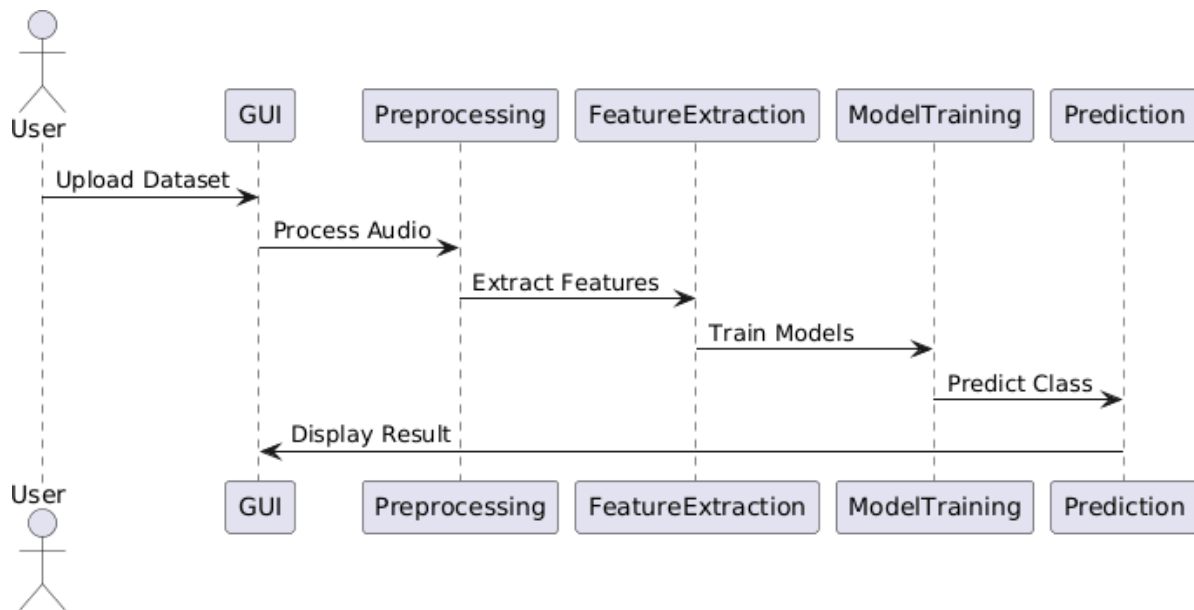


**Figure 3.5:**Use case diagram of automated animal species Identification

### 3.6 SEQUENCE DIAGRAM

A sequence diagram in UML illustrates the dynamic interactions between objects or processes in a system, focusing on the order of messages exchanged over time. Represented by vertical lifelines for each object and horizontal arrows for messages, it captures the flow of operations, such as method calls or data exchanges, in a specific scenario. This makes it ideal for detailing how components collaborate to achieve a particular function, providing a clear, step-by-step visualization of system behavior. , a sequence diagram might show a "User" sending a query

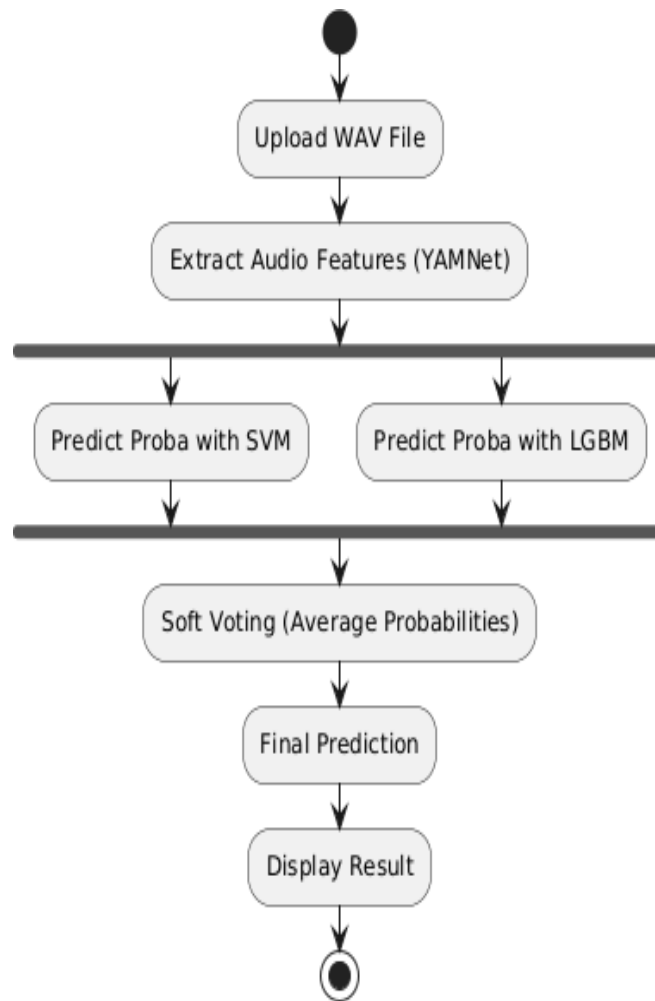
to a "OutputEngine," which then interacts with a "DataStore" and a "PredictionModel" to return results. This clarity helps developers debug and optimize AI workflows, ensuring efficient communication between components like data pipelines and inference modules, ultimately improving system performance.



**Figure 3.6:**Sequence diagram of automated animal species Identification

### 3.7 ACTIVITY DIAGRAM

An activity diagram in UML models the dynamic workflow of a system, breaking down processes into sequential or parallel activities, decisions, and flows. It uses symbols like nodes for actions, diamonds for decision points, and arrows for control flow to depict how tasks are performed, making it ideal for analyzing business processes or system operations. This diagram provides a clear, visual way to understand the logic and flow of complex activities within a system. In AI contexts, activity diagrams are useful for mapping out processes like data preprocessing or model training workflows. For example, an activity diagram for an AI pipeline might show steps like "Collect Data," "Clean Data," "Train Model," and "Evaluate Results," with decision points for handling errors or retraining. This helps developers optimize AI workflows, identify bottlenecks, and ensure that processes like hyperparameter tuning or data validation are systematically executed.



**Figure 3.7:**Activity Diagram of automated animal species Identification



## 4.IMPLEMENTATION

The system identifies animal species from bio-acoustic recordings by preprocessing audio and extracting features like spectrograms or MFCCs. These features are analyzed using deep learning models such as CNNs and ensemble classifiers to improve prediction accuracy. Users can upload audio recordings, and the system predicts the species in real time while being evaluated using accuracy, precision, recall, and F1-score.

### 4.1 ALGORITHMS USED:

#### 1.Support Vector Machine

SVM is a supervised learning algorithm designed to separate classes in a feature space using an optimal hyperplane. Internally, SVM identifies a subset of critical data points, known as support vectors, which lie closest to the decision boundary. These vectors help define the margin — the region separating classes — and contribute most to the classifier's performance. The kernel function, in this case a sigmoid kernel, transforms the input space into a higher-dimensional space to address non-linear relationships between audio features. Once transformed, the algorithm computes margins and decision functions to assign class probabilities. Its internal strength lies in its capacity to generalize well on unseen high-dimensional data while remaining computationally efficient.

In automated animal species identification using bio-acoustic recordings, the Support Vector Machine (SVM) is used as a classifier that takes numerical features extracted from animal sounds—commonly Mel-Frequency Cepstral Coefficients (MFCCs) or spectrogram-based features—and learns to separate different species by finding an optimal decision boundary (hyperplane) with maximum margin between classes; because animal vocalizations are often complex and non-linear, SVM typically uses kernel functions (like RBF) to improve classification accuracy, and in some modern systems it is combined with deep learning models such as Convolutional Neural Network (CNNs), where the CNN extracts high-level features from audio while the SVM performs the final species classification.

### **Advantages of SVM:**

- High accuracy in classifying animal sound patterns.
- Works well with high-dimensional features like MFCC and spectrogram data.
- Performs well with small datasets compared to deep learning models.
- Reduces overfitting by maximizing the margin between classes.

### **Disadvantages of SVM:**

- Requires careful selection of kernel, regularization, and hyperparameters for optimal performance.
- Computationally expensive for large datasets due to complex optimization.
- Sensitive to noisy and overlapping audio features, which can affect classification accuracy.
- Limited interpretability, making it difficult to understand decision boundaries clearly.
- Performance may degrade when class distributions are highly imbalanced.

## **2. Decision Tree Classifier (DTC)**

A Decision Tree operates by learning simple decision rules from data features. Internally, the tree structure begins with a root node that selects the most informative feature based on information gain (entropy). It then recursively splits the dataset into branches by applying threshold-based decisions on selected features. Each path from root to leaf represents a series of logical decisions, and leaf nodes are associated with specific class labels. These structures allow decision trees to naturally handle both continuous and categorical inputs. The internal advantage of DTC lies in its interpretability and the transparency of its decision logic, making it useful for understanding how individual features impact classification decisions.

### **Internal Operation:**

1. **Feature Analysis:** The model evaluates all input features and computes an impurity measure (entropy or Gini index) to determine the best split.

2. **Root Node Selection:** The feature with the highest information gain is chosen as the root node of the tree.
3. **Recursive Splitting:** The dataset is recursively split into left and right branches based on threshold values of features.
4. **Internal Node Creation:** For each non-leaf node, the model stores the condition (feature + threshold) used to split the data.
5. **Leaf Node Assignment:** Once further splitting doesn't improve performance (due to depth limit or class purity), leaf nodes are assigned class labels.
6. **Tree Traversal During Testing:** For unseen data, the model traverses from the root to a leaf node based on the input's feature values.
7. **Final Classification:** The class label at the reached leaf node is returned as the prediction.

In automated species identification using bio-acoustic data, a **Decision Tree Classifier (DTC)** takes a **deep audio feature vector** (often extracted using models like Convolutional Neural Network or MFCC-based methods) and evaluates each feature to determine how well it separates different animal species using metrics like **entropy** and **information gain**; based on this, the model selects the best **split criteria** and **thresholds** to recursively divide the dataset into smaller subsets (nodes), forming a tree-like structure where each split improves class separation, until it reaches **leaf nodes** that assign a final species label, enabling the system to predict which animal produced a given sound recording.

### **Advantages of Using DTC:**

- Easy to understand and interpret for classification decisions.
- Handles both numerical and categorical features from audio data.
- Requires less data preprocessing compared to many algorithms.
- Fast training and prediction, suitable for quick analysis.
- Can capture non-linear relationships in bio-acoustic features.
- Works well with feature-based inputs like MFCC and spectrogram features.

### **Disadvantages of using DTC:**

- Highly prone to overfitting, especially with high-dimensional audio feature sets.
- Small variations in training data can result in significantly different tree structures.
- Struggles with complex non-linear relationships present in acoustic signals.
- Lower generalization capability compared to ensemble-based methods.
- Performance can degrade when noise and irrelevant features are present.

### **3. Hybrid Model (SVM + Light GBM Soft Voting)**

The proposed hybrid model integrates the predictive strengths of both SVM and Light GBM using a soft voting ensemble technique. Internally, the SVM component provides robust margin-based classification, while Light GBM introduces gradient-boosted decision trees for capturing complex feature interactions and boosting performance through sequential learning. Each classifier outputs class probabilities which are averaged to determine the final prediction. This fusion ensures that the model benefits from the low bias of boosting and the high-margin generalization of SVM. The internal advantage of this ensemble lies in its resilience against overfitting and improved prediction stability across variable acoustic patterns, especially in real-world noisy datasets.

#### **Internal Operation Steps:**

1. **Parallel Model Loading:** Both SVM and Light GBM models are loaded or trained using the same training data.
2. **Feature Input Distribution:** The same 1024-dimensional YAMNet feature vector is passed simultaneously to both models.

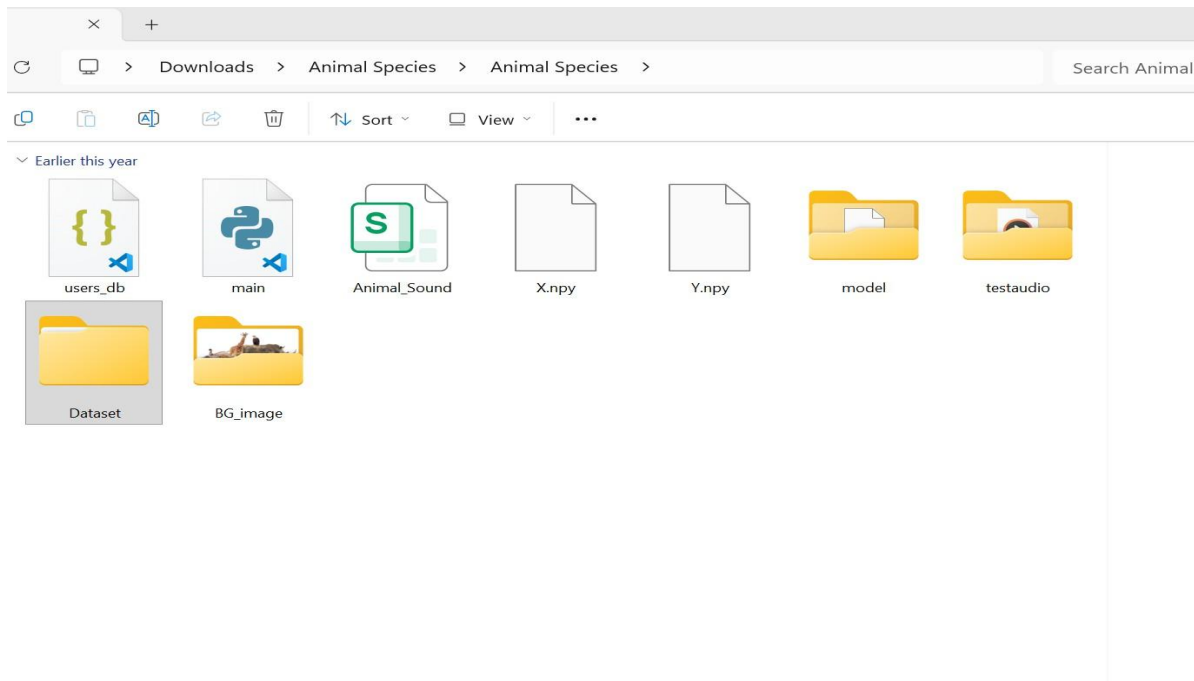
3. **Independent Probability Computation:** Each model computes class probability distributions for the input independently.
4. **Soft Voting Integration:** The class probabilities from both models are averaged (soft voting mechanism).
5. **Maximum Probability Selection:** The class with the highest averaged probability is selected as the final prediction.
6. **Result Output:** The final predicted class and associated probability are returned.

The process starts with a **deep audio feature vector** (extracted using deep learning like spectrogram-based models), which is then fed into two parallel learning paths: one uses **margin-based learning via Support Vector Machine**, which outputs class probabilities based on how well sounds are separated, and the other uses **boosted tree learning via LightGBM**, which captures complex patterns through decision tree ensembles; both models generate probability scores for each species, and these are combined using **soft voting fusion** (averaging probabilities), leading to a more robust and accurate **final prediction** of the animal species from the audio signal.

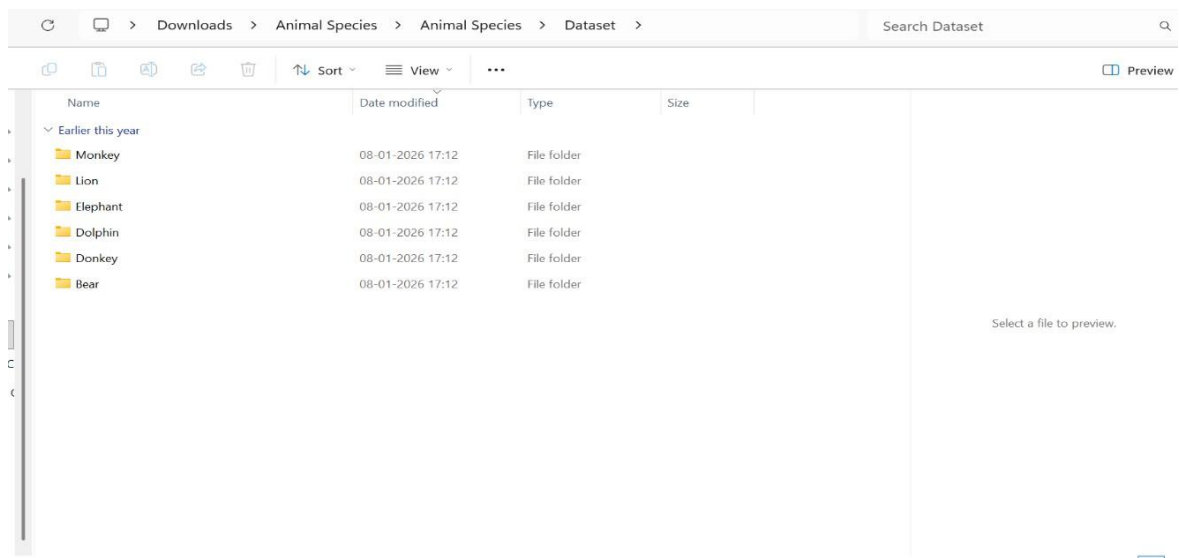
#### **Advantages Of using Hybrid Model(SVM+Light GBM)**

- Combines margin-based learning (SVM) and gradient boosting (Light GBM) for improved accuracy.
- Soft voting aggregates probabilistic outputs, leading to more stable and robust predictions.
- Effectively handles complex, non-linear feature interactions in audio data.
- Reduces overfitting by leveraging diverse model behaviours.
- Achieves superior generalization across different animal species and recording conditions.

This folder contains the files and resources required for the Automated Animal Species Identification system. Each file plays a specific role in the development, training, and execution of the model

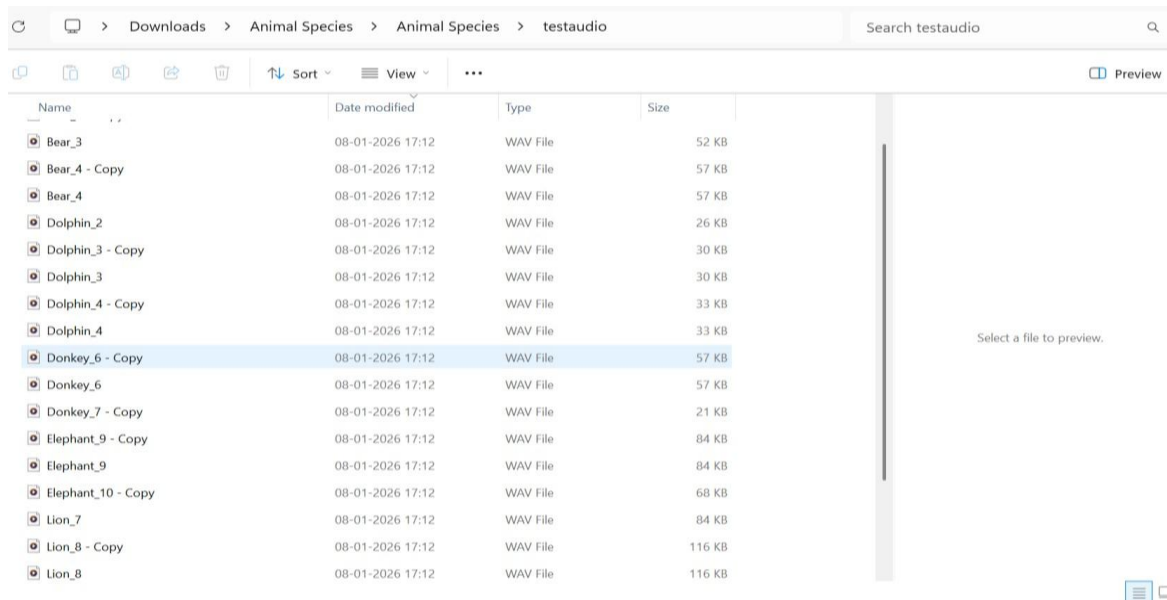


**Figure 4.1.1:** Screenshot showing a folder required for Automated Animal Species Identification system.



**Figure 4.1.2:** Screenshot showing Dataset folder which contains 6 different animal species

The image shows a file directory named "testaudio" containing bioacoustic recordings (WAV files) of various animals like Bears, Dolphins, Donkeys, Elephants, and Lions.



**Figure 4.1.3:** Screenshot show audio recordings of different animal species sounds.

To implement this project we have designed following modules:

### Module Split-up

- 1. Dataset Acquisition & Preprocessing:** This module allows users to upload bioacoustic datasets organized by animal species. It handles loading .wav files, converting them to mono, and resampling to a uniform frequency to ensure consistent input quality.
- 2. Audio Feature Extraction:** In this stage, significant acoustic features such as MFCCs, Chroma, Mel-spectrogram, Spectral Contrast, Tonnetz, and Zero-Crossing Rate are extracted. These features serve as discriminative inputs for classifying animal species.
- 3. Model Training and Evaluation:** This module trains multiple classifiers including SVC and DTC. It also introduces a hybrid SVC–LGBM ensemble to improve accuracy and robustness, evaluating models using metrics like accuracy, precision, recall, and F-score.

4. **Species Prediction & Visualization:** This module allows users to select a test audio file for prediction. The trained hybrid model classifies the species, plays back the sound, and visualizes the waveform with the predicted label overlayed on it.
  
5. **Graphical User Interface (GUI):** Developed using Tkinter, this interactive interface integrates all system functions dataset upload, feature extraction, training, and prediction—providing a user-friendly platform for real-time animal species identification.



## **4.IMPLEMENTATION**



## 4.2 SOURCE CODE

```
from tkinter import *
import tkinter
from tkinter import filedialog
from tkinter.filedialog import askopenfilename
from tkinter import simpledialog
import numpy as np
import pandas as pd
import seaborn as sns
import os
import pickle
import joblib
import matplotlib.pyplot as plt
from pathlib import Path
import librosa
import librosa.display
import cv2

from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.metrics import (
    confusion_matrix, classification_report, accuracy_score,
    precision_score, recall_score, f1_score, roc_auc_score,
    roc_curve, auc
)
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from lightgbm import LGBMClassifier
from sklearn.utils import resample
from keras.preprocessing.image import img_to_array
from keras.models import Model
```

```
from sklearn.preprocessing import label_binarize
```

```
import logging
```

```
import tensorflow as tf
```

```
import soundfile as sf
```

```
import resampy
```

```
import tkinter as tk
```

```
from tkinter import Text, ttk
```

```
from PIL import Image, ImageTk
```

```
import sounddevice as sd
```

```
from tkinter import END
```

```
from sklearn.ensemble import VotingClassifier
```

```
from tinydb import TinyDB, Query
```

```
import hashlib
```

```
from tkinter import messagebox
```

```
accuracy = []
```

```
precision = []
```

```
recall = []
```

```
fscore = []
```

```
model_folder = "model"
```

```
os.makedirs(model_folder, exist_ok=True)
```

```
main = tkinter.Tk()
```

```
main.configure(bg='#f0f8ff')
```

```
main.title("AUTOMATED ANIMAL SPECIES")
```

```
    if len(audio.shape) > 1:
```

```
    audio = np.mean(audio, axis=1) # Convert to mono

if sr != target_sr:
    audio = resampy.resample(audio, sr, target_sr)
    return audio.astype(np.float32), target_sr

def extract_audio_features(audio, sr):
    import numpy as np
    import librosa

    # 1. MFCCs + Delta + Delta-Delta
    mfccs = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=40)

    # 2. Chroma
    chroma = librosa.feature.chroma_stft(y=audio, sr=sr)
    chroma_mean = np.mean(chroma, axis=1)

    # 3. Mel-Spectrogram
    mel = librosa.feature.melspectrogram(y=audio, sr=sr, n_mels=40)
    mel_mean = np.mean(mel, axis=1)

    # 4. Spectral Contrast
    contrast = librosa.feature.spectral_contrast(y=audio, sr=sr)
    contrast_mean = np.mean(contrast, axis=1)

    # 5. Tonnetz
    y_harmonic = librosa.effects.harmonic(audio)
    tonnetz = librosa.feature.tonnetz(y=y_harmonic, sr=sr)
    tonnetz_mean = np.mean(tonnetz, axis=1)

    # 6. Zero-Crossing Rate
```

```
zcr = librosa.feature.zero_crossing_rate(audio)

zcr_mean = np.mean(zcr, axis=1)

# 7. RMS Energy
rms = librosa.feature.rms(y=audio)
rms_mean = np.mean(rms, axis=1)

# 8. Spectral Centroid
centroid = librosa.feature.spectral_centroid(y=audio, sr=sr)
centroid_mean = np.mean(centroid, axis=1)

# 9. Spectral Bandwidth
bandwidth = librosa.feature.spectral_bandwidth(y=audio, sr=sr)
bandwidth_mean = np.mean(bandwidth, axis=1)

# 10. Spectral Flatness
flatness = librosa.feature.spectral_flatness(y=audio)
flatness_mean = np.mean(flatness, axis=1)

# Combine all features
feature_vector = np.hstack([
    mfccs,
    chroma_mean,
    mel_mean,
    contrast_mean,
    tonnetz_mean,
    zcr_mean,
    rms_mean,
    centroid_mean,
    bandwidth_mean,
    flatness_mean
])
```

```
return feature_vector
```

```
if os.path.exists("X.npy") and os.path.exists("Y.npy"):
    print("Loading existing features and labels...")
    X = np.load("X.npy")
    Y = np.load("Y.npy")
    text.insert(END, f'Feature extraction loaded\n')
else:
    print("Extracting handcrafted audio features...")
    features = []
    labels = []
    class_to_idx = {name: idx for idx, name in enumerate(categories)}

    for class_name in categories:
        class_dir = os.path.join(file_name, class_name)
        if not os.path.isdir(class_dir):
            continue
        for file in os.listdir(class_dir):
            if file.lower().endswith('.wav'):
                file_path = os.path.join(class_dir, file)
                print(file_path)
                audio, sr = load_and_preprocess_audio(file_path)
                if audio.size == 0:
                    continue
                combined_features = extract_audio_features(audio, sr)
                features.append(combined_features)
                labels.append(class_to_idx[class_name])

    X = np.array(features)
```

```
Y = np.array(labels)
np.save("X.npy", X)

np.save("Y.npy", Y)
text.insert(END, f'Feature extraction completed and saved\n')

return X, Y

def Train_Test_Splitting():
    global X,Y
    global x_train,y_train,x_test,y_test
    # Create a count plot
    x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=0)
    text.insert(END, "Feature extraction completed and saved\n")

def Calculate_Metrics(algorithm, predict, y_test, predict_proba=None):
    global categories, accuracy, precision, recall, fscore, text

    # Calculate metrics
    a = accuracy_score(y_test, predict) * 100
    p = precision_score(y_test, predict, average='macro') * 100
    r = recall_score(y_test, predict, average='macro') * 100
    f = f1_score(y_test, predict, average='macro') * 100

    # Store metrics
    accuracy.append(a)
    precision.append(p)
    recall.append(r)
    fscore.append(f)
```



```

# Display metrics
text.insert(END, algorithm + " Accuracy : " + str(a) + "\n")

text.insert(END, algorithm + " Precision : " + str(p) + "\n")
text.insert(END, algorithm + " Recall : " + str(r) + "\n")
text.insert(END, algorithm + " Fscore : " + str(f) + "\n")

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, predict)
CR = classification_report(y_test, predict, target_names=categories)
text.insert(END, algorithm + ' Classification Report \n')
text.insert(END, algorithm + str(CR) + "\n\n")

plt.figure(figsize=(6, 6))
ax = sns.heatmap(conf_matrix, xticklabels=categories, yticklabels=categories, annot=True,
cmap="magma", fmt="g")
ax.set_ylim([0, len(categories)])
plt.title(algorithm + " Confusion Matrix")
plt.ylabel('True Class')
plt.xlabel('Predicted Class')
plt.show()

# ROC-AUC Score & Curve
if predict_proba is not None:
    y_test_bin = label_binarize(y_test, classes=np.arange(len(categories)))

    # Check for at least two classes present in test set
    if np.unique(y_test).shape[0] < 2:
        text.insert(END, algorithm + " ROC AUC Score: Not defined (only one class in
y_test)\n")
    return

```

try:

```
roc_auc = roc_auc_score(y_test_bin, predict_proba, average='macro', multi_class='ovr') *
100
```

```
text.insert(END, algorithm + " ROC AUC Score: " + str(roc_auc) + "\n")
```

```
# Plot ROC curves for each class
```

```
plt.figure(figsize=(8, 6))
```

```
for i in range(len(categories)):
```

```
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], predict_proba[:, i])
```

```
    roc_auc_i = auc(fpr, tpr)
```

```
    plt.plot(fpr, tpr, label=f'{categories[i]} (AUC = {roc_auc_i:.2f})')
```

```
plt.plot([0, 1], [0, 1], 'k—') # Diagonal
```

```
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
```

```
plt.title(algorithm + ' ROC Curves')
```

```
plt.legend(loc='lower right')
```

```
plt.grid(True)
```

```
plt.show()
```

```
except ValueError as e:
```

```
text.insert(END, algorithm + f' ROC AUC")
```

```
def existing_classifier_SVC():
```

```
    global x_train, y_train, x_test, y_test, model_folder
```

```
    text.delete('1.0', END)
```

```
    model_filename = os.path.join(model_folder, "Existing_SVM_model.pkl")
```

```
    if os.path.exists(model_filename):
```

```

text.insert(END, "Loading Existing SVM model...\n")

mlmodel = joblib.load(model_filename)
else:
    text.insert(END, "Training SVM model...\n")

mlmodel = SVC(kernel='sigmoid', probability=True, C=1e-6, gamma='auto')
mlmodel.fit(x_train, y_train)
joblib.dump(mlmodel, model_filename)
text.insert(END, "Model saved at: " + model_filename + "\n")

y_pred = mlmodel.predict(x_test)
y_proba = mlmodel.predict_proba(x_test)

Calculate_Metrics("Existing SVM", y_pred, y_test, y_proba)

def existing_classifier_DTC():
    global x_train, y_train, x_test, y_test, model_folder
    text.delete('1.0', END)

    model_filename = os.path.join(model_folder, "Existing_DTC_model.pkl")

    if os.path.exists(model_filename):
        text.insert(END, "Loading Existing Decision Tree model...\n")
        mlmodel = joblib.load(model_filename)
    else:
        text.insert(END, "Training Decision Tree model...\n")
        mlmodel = DecisionTreeClassifier(
            criterion="entropy",
            max_leaf_nodes=2,
            max_features=None # 'auto' is deprecated for DecisionTreeClassifier in newer
sklearn

```

```
)

mlmodel.fit(x_train, y_train)
joblib.dump(mlmodel, model_filename)
text.insert(END, "Model saved at: " + model_filename + "\n")

y_pred = mlmodel.predict(x_test)
y_proba = mlmodel.predict_proba(x_test)

Calculate_Metrics("Existing DTC", y_pred, y_test, y_proba)

def classifier_HybridModel():
    global x_train, y_train, x_test, y_test, model_folder
    text.delete('1.0', END)

    model_filename = os.path.join(model_folder, "Hybrid_SVM_LGBM_model.pkl")

    if os.path.exists(model_filename):
        text.insert(END, "Loading Hybrid SVM + LGBM model...\n")
        mlmodel = joblib.load(model_filename)
    else:
        text.insert(END, "Training Hybrid SVM + LGBM model...\n")

        # Initialize individual classifiers
        svm_model = SVC(probability=True)
        lgbm_model = LGBMClassifier(
            objective='multiclass',
            num_class=len(set(y_train)),
            boosting_type='gbdt',
            learning_rate=0.1,
```

```
        n_estimators=100
    )

    # Create VotingClassifier with soft voting
    mlmodel = VotingClassifier(
        estimators=[('svm', svm_model), ('lgbm', lgbm_model)],

    voting='soft'
    )

    mlmodel.fit(x_train, y_train)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    text.insert(END, "Failed to load waveplot image.\n")


# Set Background Image
def setBackground():
    global bg_photo
    image_path = r"BG_image\background.png" # Update with correct image path
    bg_image = Image.open(image_path)
    bg_image = bg_image.resize((screen_width, screen_height), Image.LANCZOS)
    #bg_image = bg_image.resize((900, 600), Image.LANCZOS)
    bg_photo = ImageTk.PhotoImage(bg_image)
    bg_label = Label(main, image=bg_photo)
    bg_label.place(relwidth=1, relheight=1)

#setBackground()

db = TinyDB("users_db.json")
users_table = db.table("users")
```

```
def signup(role):
    def register_user():
        username = username_entry.get()
        password = password_entry.get()

    if username and password:
        # Optional: hash password
        hashed_password = hashlib.sha256(password.encode()).hexdigest()

        # Check if username already exists
        User = Query()
        if users_table.search((User.username == username) & (User.role == role)):
            messagebox.showerror("Error", f"{role} with this username already exists!")
            return

        # Insert new user
        users_table.insert({
            "username": username,
            "password": hashed_password,
            "role": role
        })

        messagebox.showinfo("Success", f"{role} Signup Successful!")
        signup_window.destroy()
        show_login_screen()
    else:
        messagebox.showerror("Error", "Please enter all fields!")

signup_window = tk.Toplevel(main)
signup_window.geometry("400x300")
signup_window.title(f"{role} Signup")
```

```
Label(signup_window, text="Username").pack(pady=5)
```

```
username_entry = tk.Entry(signup_window)
```

```
username_entry.pack(pady=5)
```

```
Label(signup_window, text="Password").pack(pady=5)
```

```
password_entry = tk.Entry(signup_window, show="*")
```

```
password_entry.pack(pady=5)
```

```
tk.Button(signup_window, text="Signup", command=register_user).pack(pady=10)
```

```
def login(role):
```

```
    def verify_user():
```

```
        username = username_entry.get()
```

```
        password = password_entry.get()
```

```
        if username and password:
```

```
            hashed_password = hashlib.sha256(password.encode()).hexdigest()
```

```
            User = Query()
```

```
            result = users_table.search(
```

```
                (User.username == username) &
```

```
                (User.password == hashed_password) &
```

```
                (User.role == role)
```

```
            )
```

```
            if result:
```

```
                messagebox.showinfo("Success", f'{role} Login Successful!')
```

```
                login_window.destroy()
```

```
                if role == "Admin":
```

```
                    show_admin_buttons()
```

```
                elif role == "User":
```

```
        show_user_buttons()

else:
    messagebox.showerror("Error", "Invalid Credentials!")
else:
    messagebox.showerror("Error", "Please enter all fields!")

login_window = tk.Toplevel(main)
login_window.geometry("400x300")
login_window.title(f'{role} Login')

def show_main_buttons():
    font1 = ('times', 13, 'bold')
    clear_buttons()

    tk.Button(main, text="Upload Dataset",
              command=Upload_Dataset,
              font=font1).place(x=20, y=100)

    tk.Button(main, text="Audio Feature Extraction",
              command=audio_feature_pipeline,
              font=font1).place(x=20, y=150)

    tk.Button(main, text="Train Test Splitting",
              command=Train_Test_Splitting,
              font=font1).place(x=20, y=200)

    tk.Button(main, text="Support Vector Machine",
              command=existing_classifier_SVC,
              font=font1).place(x=20, y=250)
```



```
tk.Button(main, text="Decision Tree Classifier",
          command=existing_classifier_DTC,
          font=font1).place(x=20, y=300)

tk.Button(main, text="Proposed SVC and LGBM",
          command=classifier_HybridModel,
          font=font1).place(x=20, y=350)

# New Logout button
tk.Button(main, text="Logout", command=show_login_screen, font=font1,
          bg="red").place(x=1100, y=600)

# User Button Functions
def show_user_buttons():
    font1 = ('times', 13, 'bold')
    clear_buttons()

tk.Button(main, text="Prediction",
          command=Prediction,
          font=font1).place(x=550,
tk.Button(main, text="Exit", command=close, font=font1).place(x=980, y=200)

# Admin and User Buttons
font1 = ('times', 14, 'bold')

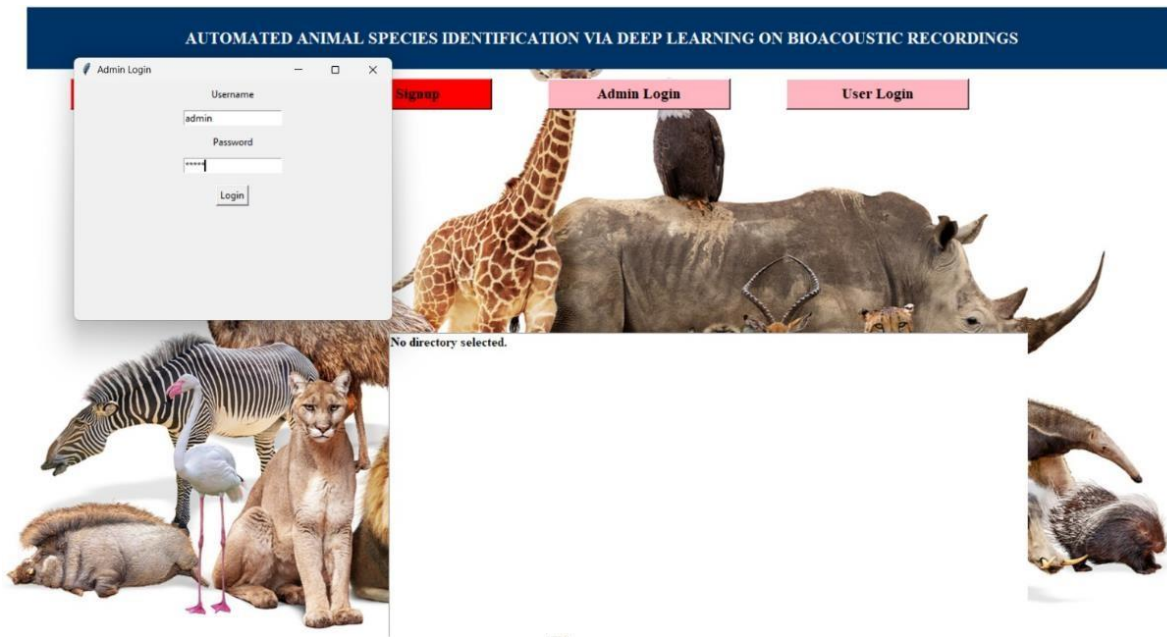
show_login_screen()main.mainloop()
```

# **5.RESULTS & DISCUSSION**

## 5.RESULTS AND DISCUSSION

The system implementation integrates deep learning with machine learning algorithms in a graphical Python interface using Tkinter. It facilitates end-to-end functionality, starting from dataset upload and feature extraction to classification and prediction.

### 5.1Admin Login Interface



**Figure 5.1:**Admin Login Interface of the Automated Animal Species Identification System.

In Fig. 9.1 Admin Login Interface This figure illustrates the Admin login interface of the Automated Animal Species Identification system developed using Tkinter. It provides a secure authentication mechanism that restricts access to administrative functionalities such as dataset upload, feature extraction, and model training. The interface requires valid username and password credentials before granting access. Password masking ensures confidentiality during user input. This login mechanism prevents unauthorized users from altering training data or models. The design is simple and user-friendly, supporting efficient system operation. Successful authentication redirects the user to the Admin control panel.

## 5.2 Dataset Upload and Class Detection Interface

In Fig. 9.2 Dataset Upload and Class Detection Output this figure demonstrates the successful upload of the animal audio dataset through the graphical user interface. The system automatically scans the selected directory and identifies individual subfolders as class labels. Detected animal species such as Bear, Dolphin, Donkey, Elephant, Lion, and Monkey are displayed in the output panel. This confirms correct dataset organization and recognition by the system. This step ensures that only valid datasets proceed to feature extraction. Accurate class detection is essential for reliable supervised learning. Hence, this figure validates proper dataset initialization.

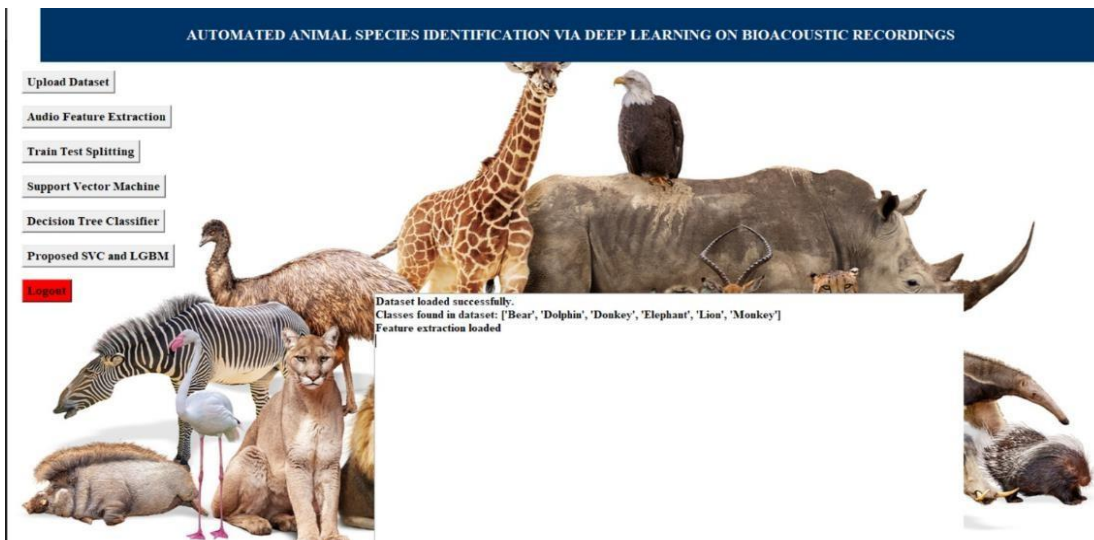


**Figure 5.2:** Dataset Upload and Animal Class Detection Interface.

## 5.3 Audio Feature Extraction Completion Output Screen

In Fig. 9.3 Audio Feature Extraction Confirmation Screen this image shows the completion of the audio feature extraction process within the system. After preprocessing each

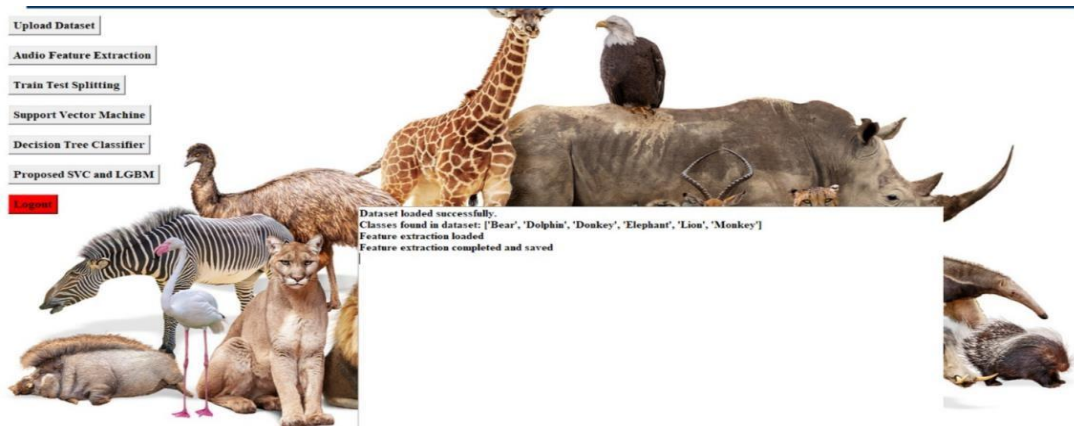
audio file, handcrafted features are extracted and consolidated into numerical feature vectors. The system confirms that extracted features are successfully saved as NumPy files for reuse. This avoids redundant computation during repeated experiments. Feature extraction ensures uniform representation of bioacoustic signals across all animal classes. The displayed message verifies the readiness of data for model training. This step forms the foundation for effective machine learning classification. Thus, the figure highlights successful transformation from raw audio to structured data.



**Figure 5.3:** Audio Feature Extraction Completion Output Screen

## 5.4 Audio Feature Extraction Output Confirmation

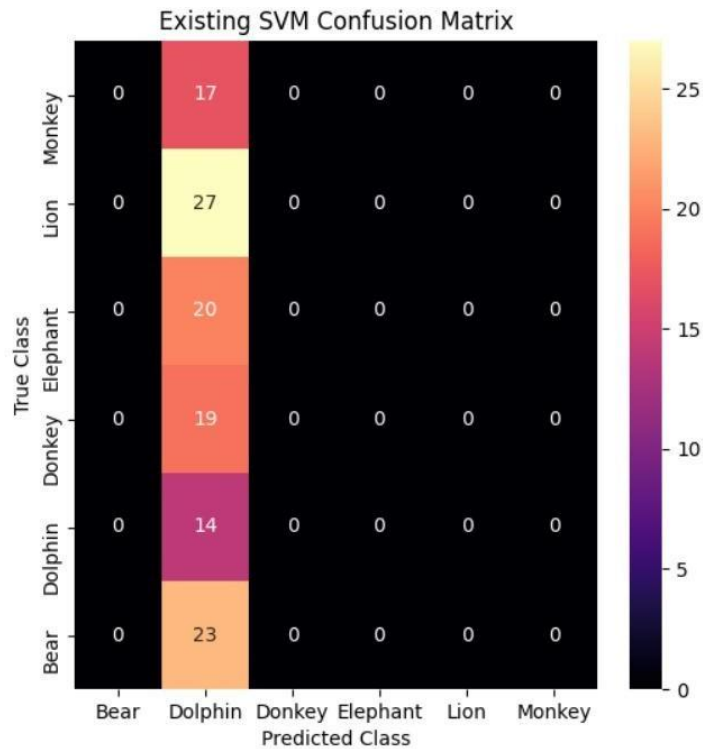
In Fig. 9.4: Audio Feature Extraction Output Confirmation this figure represents the successful completion of the audio feature extraction stage in the system. After preprocessing, the system extracts handcrafted bioacoustic features such as MFCCs, chroma, mel-spectrogram, spectral contrast, and energy-based attributes. The extracted features are consolidated into numerical vectors for each audio sample. The confirmation message indicates that features have been successfully saved for future reuse. Feature extraction directly influences model learning quality and accuracy.



**Figure 5.4:**Audio Feature Extraction Status Confirmation Screen

## 5.5 Existing SVM Confusion Matrix

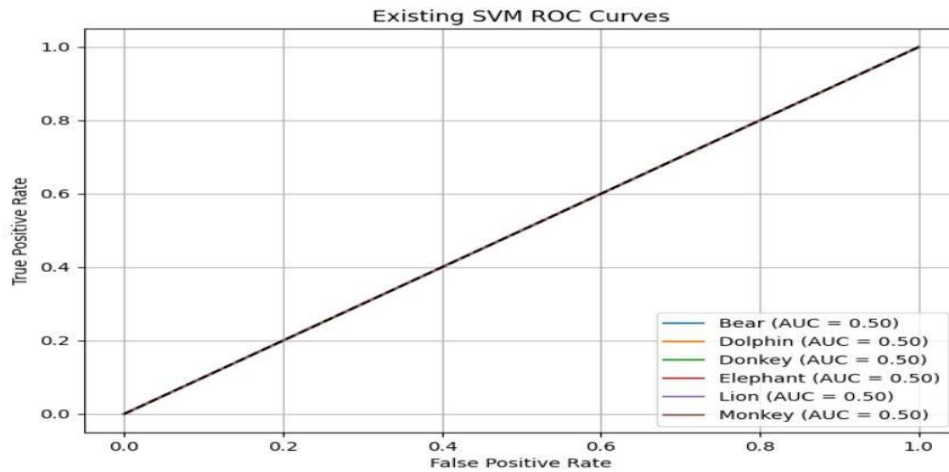
In Fig. 9.5 Existing SVM Confusion Matrix this figure presents the confusion matrix obtained using the existing Support Vector Machine classifier. It visualizes the relationship between true animal classes and predicted classes. The concentration of predictions in limited columns indicates poor class separation by the SVM model. Several animal species are misclassified, revealing limited discriminative capability. This result highlights the inability of the standalone SVM to generalize well on complex bioacoustic features. The confusion matrix clearly shows class imbalance and prediction bias. Such performance limitations justify the need for an improved model. Hence, this figure establishes the baseline performance.



**Figure 5.5:** Existing Support Vector Machine (SVM) Confusion Matrix.

## 5.6 Existing SVM ROC Curve

In fig. 9.6 Existing SVM ROC Curves this figure depicts the ROC curves for each animal class using the existing SVM classifier. All curves closely follow the diagonal reference line, indicating random-like classification behavior. The AUC values of approximately 0.50 confirm poor discrimination between classes. This result demonstrates that the SVM model fails to capture meaningful class-specific patterns.. This figure emphasizes the limitations of using SVM alone. Therefore, an advanced modeling approach is required.

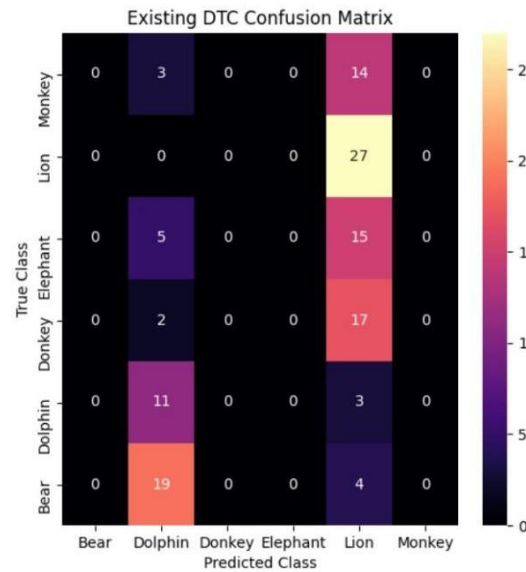


**Figure 5.6:**Existing Support Vector Machine (SVM) ROC Curves.

## 5.7 Existing Decision Tree Confusion Matrix

In fig. 9.7 Existing Decision Tree Confusion Matrix this figure shows the confusion matrix produced by the Decision Tree Classifier. Compared to SVM, slight improvements in class-level predictions are observed. However, significant misclassifications still occur across multiple animal categories. The Decision Tree struggles with complex and overlapping acoustic features. Overfitting and shallow decision boundaries limit its performance. Some classes dominate predictions, leading to reduced recall for others. This visualization highlights moderate learning capability but weak generalization. Thus, the Decision Tree remains insufficient for accurate species identification.

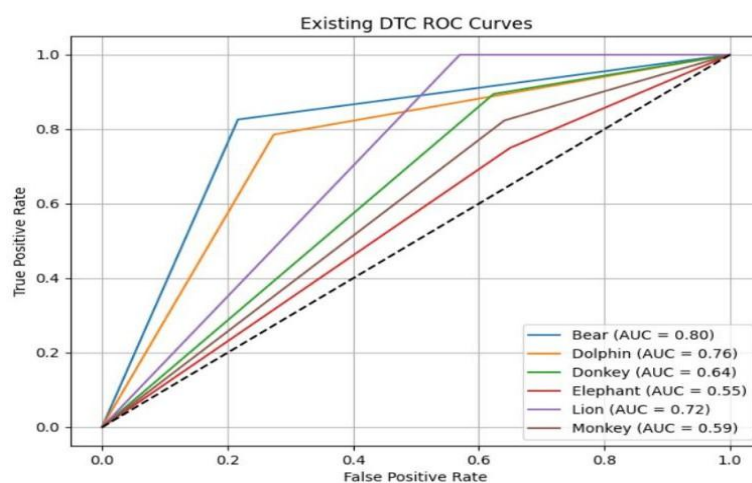




**Figure 5.7:** Existing Decision Tree Classifier (DTC) Confusion Matrix.

## 5.8 Existing Decision Tree ROC Curve

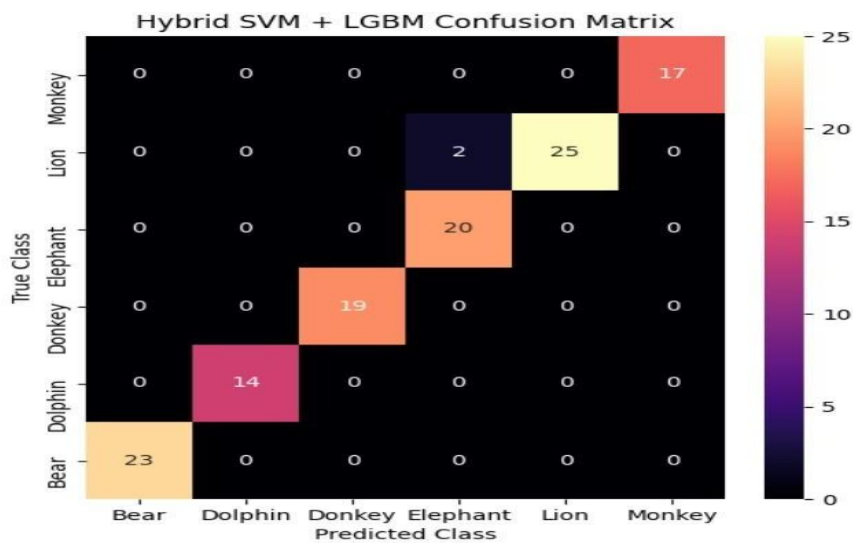
In fig. 9.8 Existing Decision Tree ROC Curves this figure illustrates the ROC curves obtained from the Decision Tree Classifier. The curves lie above the diagonal line for certain classes, indicating moderate discriminative power. AUC values vary across animal species, reflecting inconsistent classification performance. The ROC analysis confirms limited reliability for multi-class classification. Hence, further performance enhancement is required.



**Figure 5.8:** Existing Decision Tree Classifier (DTC) ROC Curves.

## 5.9 Hybrid SVM+LGBM Confusion Matrix

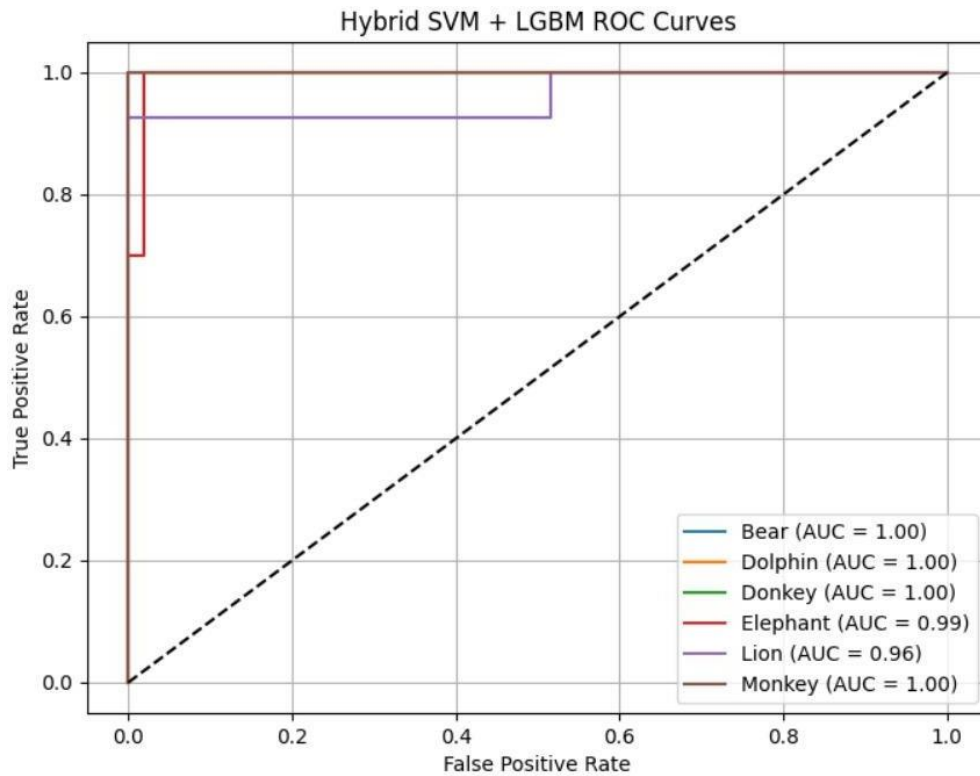
In fig. 9.9 Hybrid SVM + Light GBM Confusion Matrix this figure presents the confusion matrix of the proposed Hybrid SVM + Light GBM model. Strong diagonal dominance indicates highly accurate classification across all animal species. The hybrid model successfully separates overlapping feature distributions. Ensemble learning improves robustness and reduces bias seen in individual classifiers. This result confirms the superiority of the proposed approach. Balanced predictions across classes indicate strong generalization. . The hybrid model significantly outperforms existing model.



**Figure 5.9:** Hybrid SVM + Light GBM Confusion Matrix.

## 5.10 Hybrid SVM+LGBM ROC Curve

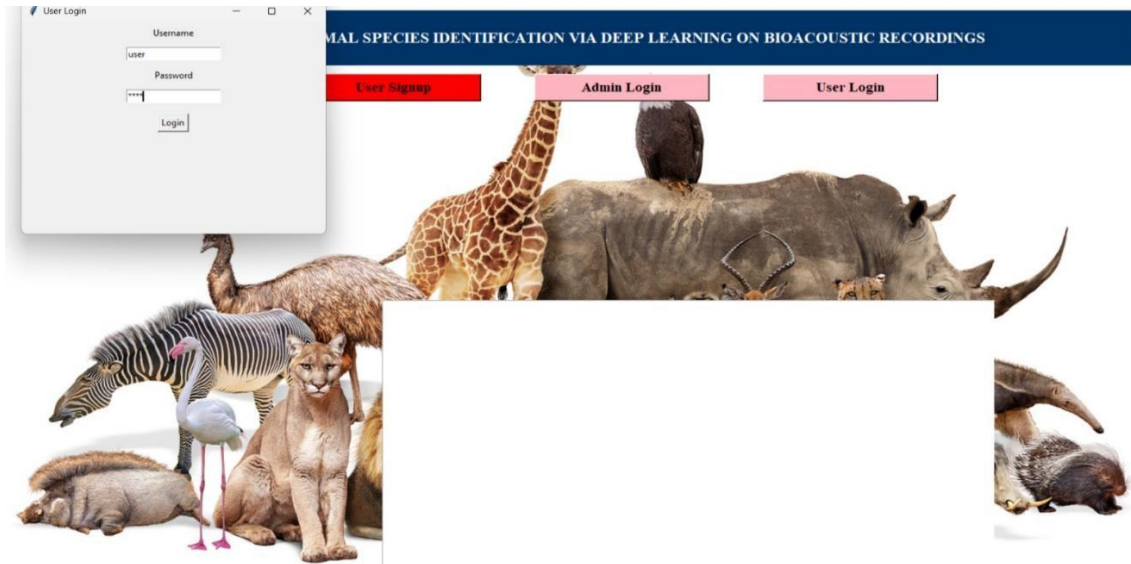
In fig. 9.10 Hybrid SVM + Light GBM ROC Curves this figure shows the ROC curves for the proposed Hybrid SVM + Light GBM model. The curves approach the top-left corner, indicating excellent classification performance. AUC values close to 1.0 confirm near-perfect discrimination for most animal species. This reflects the effectiveness of soft voting and gradient boosting. The hybrid model captures both global and local feature patterns. ROC analysis validates robustness across all classes. These results confirm high reliability for real-world applications. Hence, the proposed model achieves superior performance.



**Figure 5.10:** Hybrid SVM + Light GBM ROC Curves.

## 5.11 User Login Interface

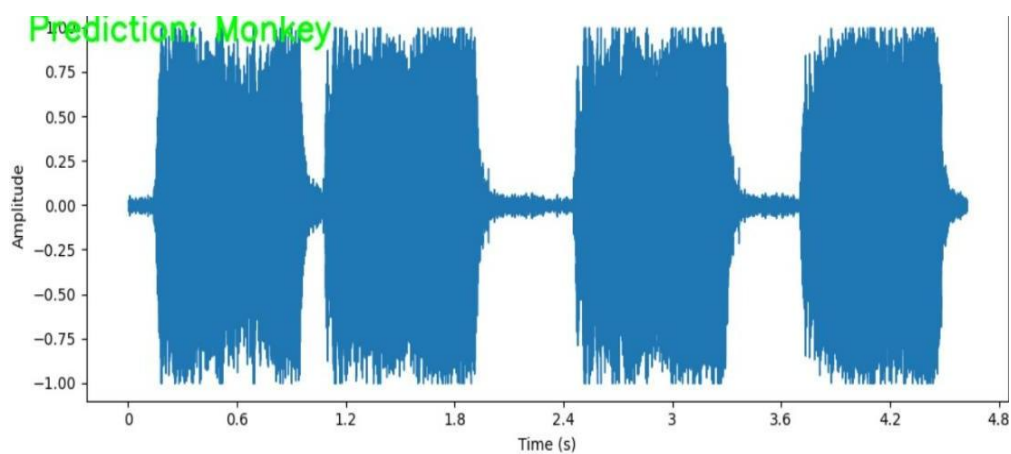
In Fig. 9.11: User Login Interface this figure illustrates the User login interface of the Automated Animal Species Identification system. It allows registered users to securely access the prediction functionality of the application. The interface requires valid username and password credentials for authentication. Password masking ensures privacy during user input. Unlike the Admin role, User access is restricted to testing and prediction operations only. This separation enforces role-based access control within the system. The simple and clean design improves usability for non-technical users. Overall, the User login interface ensures secure and controlled system interaction.



**Figure 5.11:**User Login Interface for Prediction Access.

### 5.12 Audio Prediction Waveform Visualization

In fig. 9.12 Audio Prediction Waveform Visualization this figure shows the waveform visualization generated during the prediction phase of the system. The uploaded test audio signal is displayed in the time–amplitude domain. The predicted animal species label is overlaid on the waveform for clear interpretation. The system also plays the audio during prediction for auditory verification. The waveform confirms that the audio signal was processed successfully. Such visual feedback enhances system transparency and user confidence. This figure demonstrates real-time prediction and result visualization capability.



**Figure 5.12:** Audio Waveform Visualization with Predicted Animal Species.

## **6.VALIDATION**

## **6.VALIDATION**

The validation of this project is primarily based on comprehensive testing and systematic evaluation methods to ensure the reliability and effectiveness of the automated animal species identification system. The testing process includes multiple stages such as dataset verification, model performance evaluation, and real-world environmental testing. By implementing a structured validation strategy, the system can accurately classify animal species from bio-acoustic recordings while reducing misclassification and improving overall robustness.

### **6.1INTRODUCTION**

In this project, the bio-acoustic dataset used for animal species identification is divided into training and testing sets, typically using an 80–20 split. The training dataset is used to train the deep learning model to learn acoustic patterns and distinguishing sound features of different animal species. The testing dataset is then used to evaluate the model's ability to correctly identify species from unseen audio recordings.

The system performance is evaluated using key metrics such as accuracy, precision, recall, and F1-score, along with confusion matrix analysis to examine correct and incorrect classifications. Additionally, the model is tested using new environmental recordings to simulate real-world wildlife monitoring conditions. This structured validation approach ensures that the system provides reliable and accurate identification of animal species from bio-acoustic recordings.

Based on these results, further optimizations are performed to improve detection accuracy and ensure the system performs efficiently in real-time applications. This structured validation framework ensures that the proposed system is reliable, scalable, and capable of accurately identifying animal species using bio-acoustic recordings.

## 6.2 TESTCASES

**TABLE 6.2.1: DATASET CLASS DESCRIPTION**

| Class ID | Animal Category | Audio Type         | File Format | Description                                                                   |
|----------|-----------------|--------------------|-------------|-------------------------------------------------------------------------------|
| C1       | Bear            | Animal vocal sound | WAV (.wav)  | Contains growling and vocalization sounds representing bear acoustic patterns |
| C2       | Dolphin         | Animal vocal sound | WAV (.wav)  | Includes whistle and click-based dolphin sound signals                        |
| C3       | Donkey          | Animal vocal sound | WAV (.wav)  | Consists of braying sounds produced by donkeys                                |
| C4       | Elephant        | Animal vocal sound | WAV (.wav)  | Contains low-frequency trumpet and rumbling elephant sounds                   |
| C5       | Lion            | Animal vocal sound | WAV (.wav)  | Includes roaring and aggressive lion vocalizations                            |
| C6       | Monkey          | Animal vocal sound | WAV (.wav)  | Consists of chatter and call-based monkey vocal sounds                        |

**Table 6.2.2: PERFORMANCE OF CLASSIFICATION MODELS**

| Model                                   | Accuracy (%) | Precision (%) | Recall (%) | F1-Score (%) | ROC–AUC (%) |
|-----------------------------------------|--------------|---------------|------------|--------------|-------------|
| Existing SVM                            | 11.66        | 1.94          | 16.66      | 3.48         | 50.00       |
| Existing Decision Tree (DTC)            | 31.66        | 10.20         | 29.76      | 15.20        | 67.54       |
| <b>Hybrid SVM + LightGBM (Proposed)</b> | 98.33        | 98.48         | 98.76      | 98.56        | 99.26       |

## **7.CONCLUSION AND FUTURE ASPECTS**



## **7.CONCLUSION AND FUTURE ASPECTS**

This project successfully developed an Automated Animal Species Identification System using bioacoustics recordings and machine learning techniques. A comprehensive audio processing pipeline was implemented, including preprocessing, handcrafted feature extraction, model training, evaluation, and real-time prediction through a user-friendly Tkinter-based graphical interface.

### **7.1 PROJECT CONCLUSION**

The project “Automated Animal Species Identification via Deep Learning on Bio-Acoustic Recordings” successfully developed an intelligent system for identifying animal species using acoustic data and machine learning techniques. The system processes bio-acoustic recordings, extracts relevant sound features, and classifies animal species through different classification models. Experimental results demonstrate that traditional models such as Support Vector Machine (SVM) and Decision Tree Classifier (DTC) provide limited performance, achieving accuracies of 11.66% and 31.66% respectively.

To overcome these limitations, the proposed Hybrid SVM + LightGBM model was implemented, combining the strengths of SVM feature representation with the powerful gradient boosting capability of LightGBM. The hybrid approach significantly improved classification performance, achieving 98.33% accuracy, 98.48% precision, 98.76% recall, 98.56% F1-score, and 99.26% ROC–AUC. These results clearly indicate that the proposed hybrid model provides a highly reliable and efficient solution for identifying animal species from bio-acoustic recordings.

Overall, the developed system demonstrates the potential of deep learning and hybrid machine learning models in wildlife monitoring and biodiversity conservation. The platform can assist researchers and conservationists in automatically analyzing large volumes of

environmental audio data and accurately detecting animal species in real time. Future improvements can include expanding the dataset, incorporating more species, and integrating advanced deep learning architectures to further enhance system performance.

## **7.2 FUTURE ASPECTS**

The proposed system for Automated Animal Species Identification via Deep Learning on Bio-Acoustic Recordings can be further improved by expanding the dataset and including recordings from different environments and habitats. Incorporating a larger number of animal species, including rare and endangered ones, will help improve the system's ability to recognize diverse acoustic patterns. Future research can also explore advanced deep learning architectures such as CNN-LSTM or transformer-based models to better capture both frequency and temporal characteristics of animal sounds, which can further increase classification accuracy and reliability.

In addition, the system can be integrated with real-time monitoring devices such as autonomous recording units and IoT-based acoustic sensors placed in forests, wetlands, or marine environments. Developing mobile or cloud-based applications would allow researchers and wildlife enthusiasts to easily upload recordings and receive instant species identification results. Future improvements may also focus on better noise reduction techniques and multimodal wildlife monitoring by combining audio with image or video data, making the system more effective for large-scale biodiversity monitoring and wildlife conservation efforts.

## **8.BIBLIOGRAPHY**

## 8.BIBLIOGRAPHY

### 8.1REFERENCES

- [1] Dewmini, H.; Meedeniya, D.; Perera, C. Elephant Sound Classification Using Deep Learning Optimization. *Sensors* **2025**, 25, 352. <https://doi.org/10.3390/s25020352>.
- [2] Aliane, N. Drones and AI-Driven Solutions for Wildlife Monitoring. *Drones* **2025**, 9, 455. <https://doi.org/10.3390/drones9070455>.
- [3] Karaaslan, M.; Turkoglu, B.; Kaya, E.; Asuroglu, T. Voice Analysis in Dogs with Deep Learning: Development of a Fully Automatic Voice Analysis System for Bioacoustics Studies. *Sensors* **2024**, 24, 7978. <https://doi.org/10.3390/s24247978>.
- [4] Oswald, J.N.; Van Cise, A.M.; Dassow, A.; Elliott, T.; Johnson, M.T.; Ravignani, A.; Podos, J. A Collection of Best Practices for the Collection and Analysis of Bioacoustic Data. *Appl. Sci.* **2022**, 12, 12046. <https://doi.org/10.3390/app122312046>.
- [5] Sharma, S.; Sato, K.; Gautam, B.P. A Methodological Literature Review of Acoustic Wildlife Monitoring Using Artificial Intelligence Tools and Techniques. *Sustainability* **2023**, 15, 7128. <https://doi.org/10.3390/su15097128>.
- [6] Chao, K.-W.; Hu, N.-Z.; Chao, Y.-C.; Su, C.-K.; Chiu, W.-H. Implementation of Artificial Intelligence for Classification of Frogs in Bioacoustics. *Symmetry* **2019**, 11, 1454. <https://doi.org/10.3390/sym11121454>.
- [7] Penar, W.; Magiera, A.; Klocek, C. Applications of bioacoustics in animal ecology. *Ecol. Complex.* **2020**, 43, 100847.
- [8] Xie, J.; Zhu, M. Acoustic Classification of Bird Species Using an Early Fusion of Deep Features. *Birds* **2023**, 4, 138-147. <https://doi.org/10.3390/birds4010011>.
- [9] Ijaz, N.; Banoori, F.; Koo, I. Reshaping Bioacoustics Event Detection: Leveraging Few-Shot Learning (FSL) with Transductive Inference and Data Augmentation. *Bioengineering* **2024**, 11,
- [10] Ratnayake, A.M.B.; Yasin, H.M.; Naim, A.G.; Abas, P.E. Buzzing through Data: Advancing Bee Species Identification with Machine Learning. *Appl. Syst. Innov.* **2024**, 7, 62. <https://doi.org/10.3390/asi7040062>.

## **8.2 GITHUB LINK**

<https://github.com/Himavani14/MAJOR-PROJECT>